

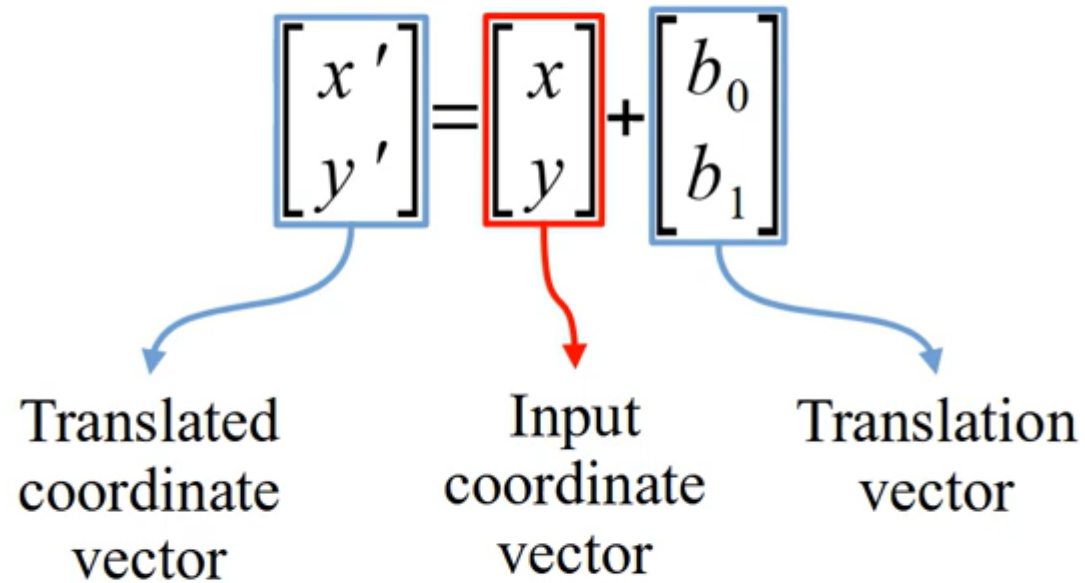
Image Transformations

Image Translation

Translation equation in Cartesian coordinates

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$$

Translation equation in Cartesian coordinates



The diagram illustrates the translation equation in Cartesian coordinates. It features three vectors represented as column matrices: $\begin{bmatrix} x' \\ y' \end{bmatrix}$, $\begin{bmatrix} x \\ y \end{bmatrix}$, and $\begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$. The equation is $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$. A blue arrow points from the first vector to the label "Translated coordinate vector". A red arrow points from the second vector to the label "Input coordinate vector". A blue arrow points from the third vector to the label "Translation vector".

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$$

Translated coordinate vector

Input coordinate vector

Translation vector

$$x' = x + b_0$$

$$y' = y + b_1$$

b_0 is translation in x direction

b_1 is translation in y direction

Translation equation in homogeneous coordinates

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & b_0 \\ 0 & 1 & b_1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

In Cartesian coordinates

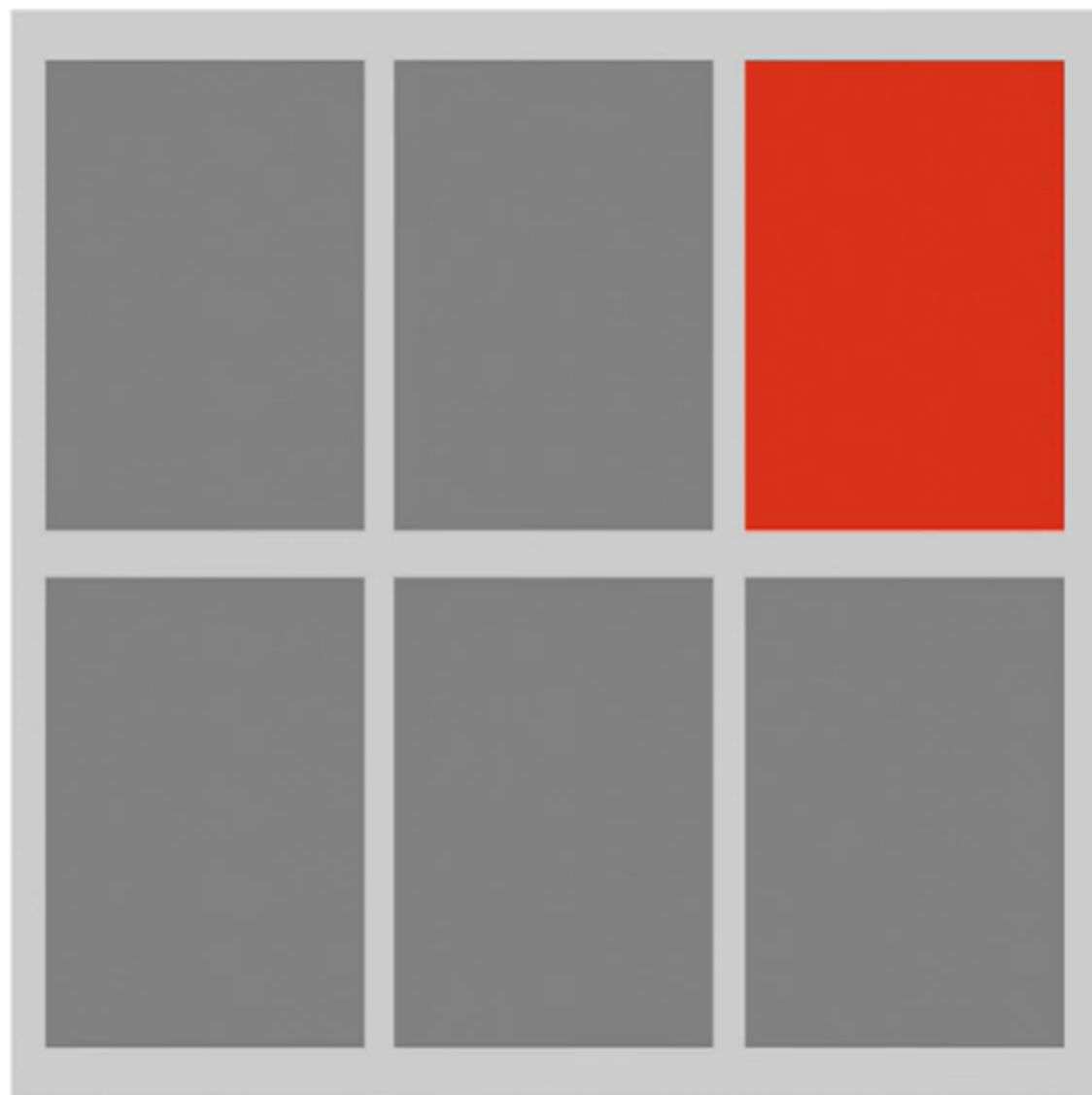
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$$

Vector addition

In homogeneous coordinates

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & b_0 \\ 0 & 1 & b_1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Matrix-vector multiplication

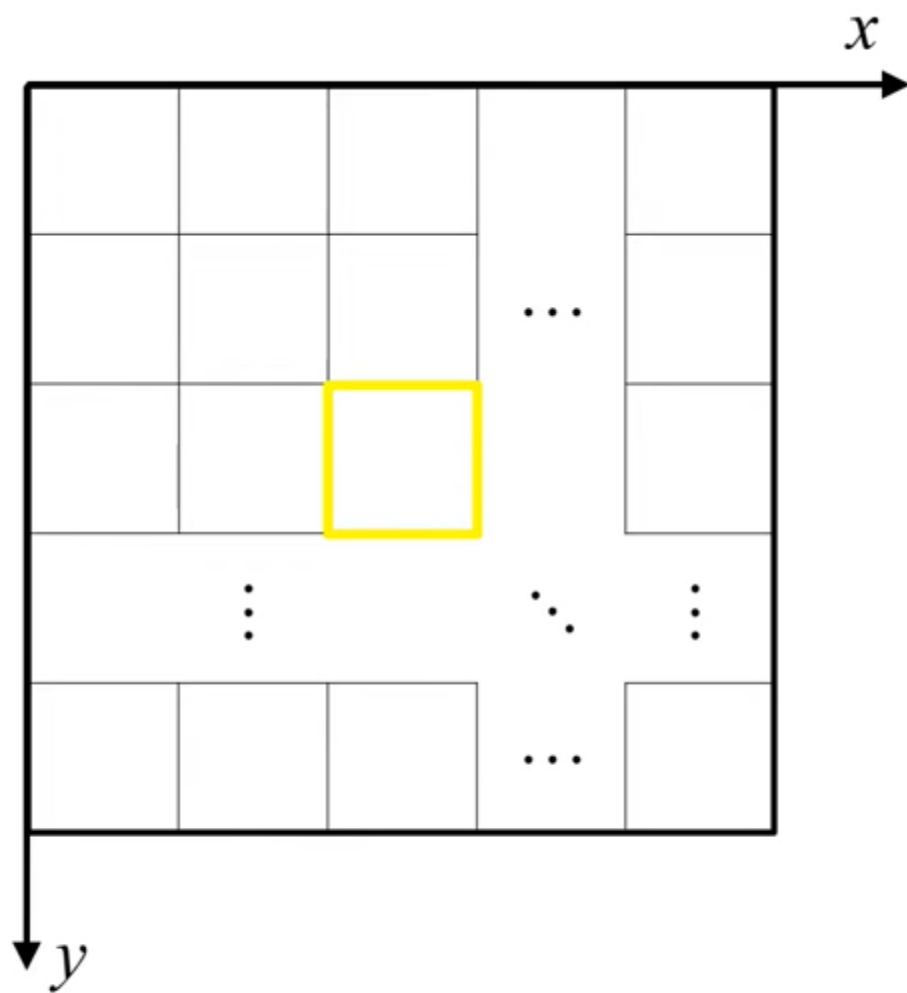


Original Image



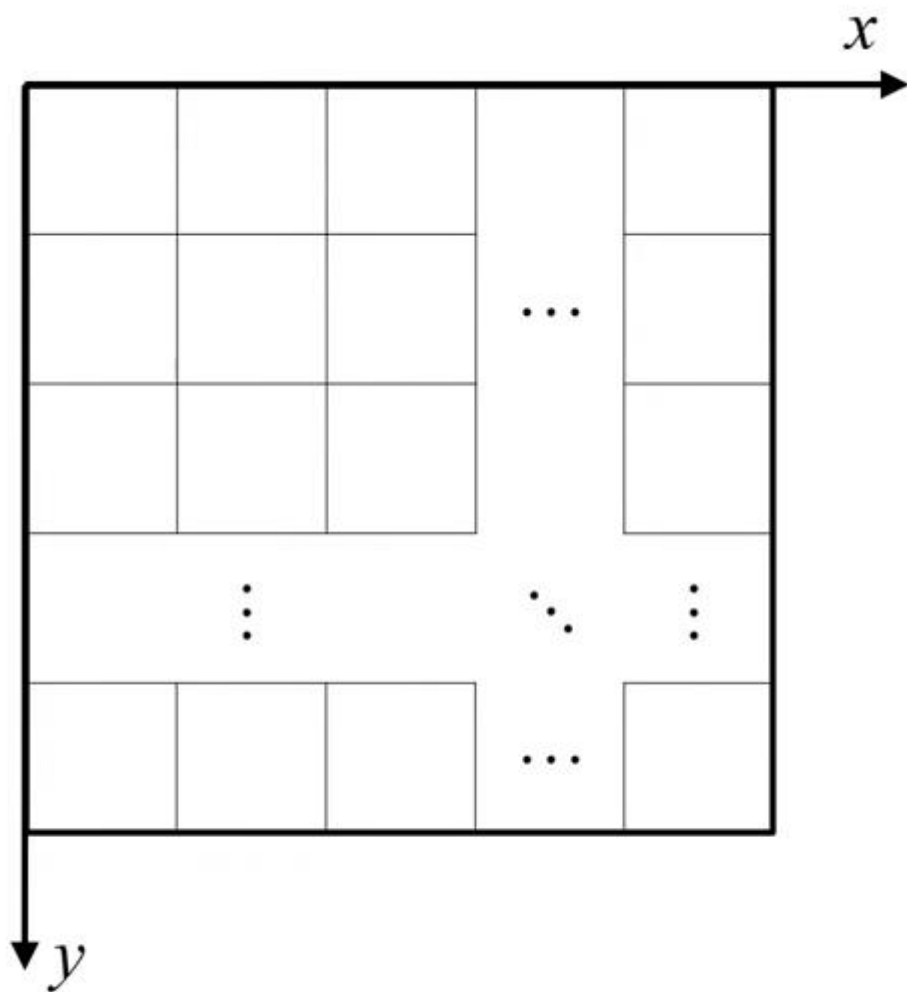
Translated Image

Affine Transformation



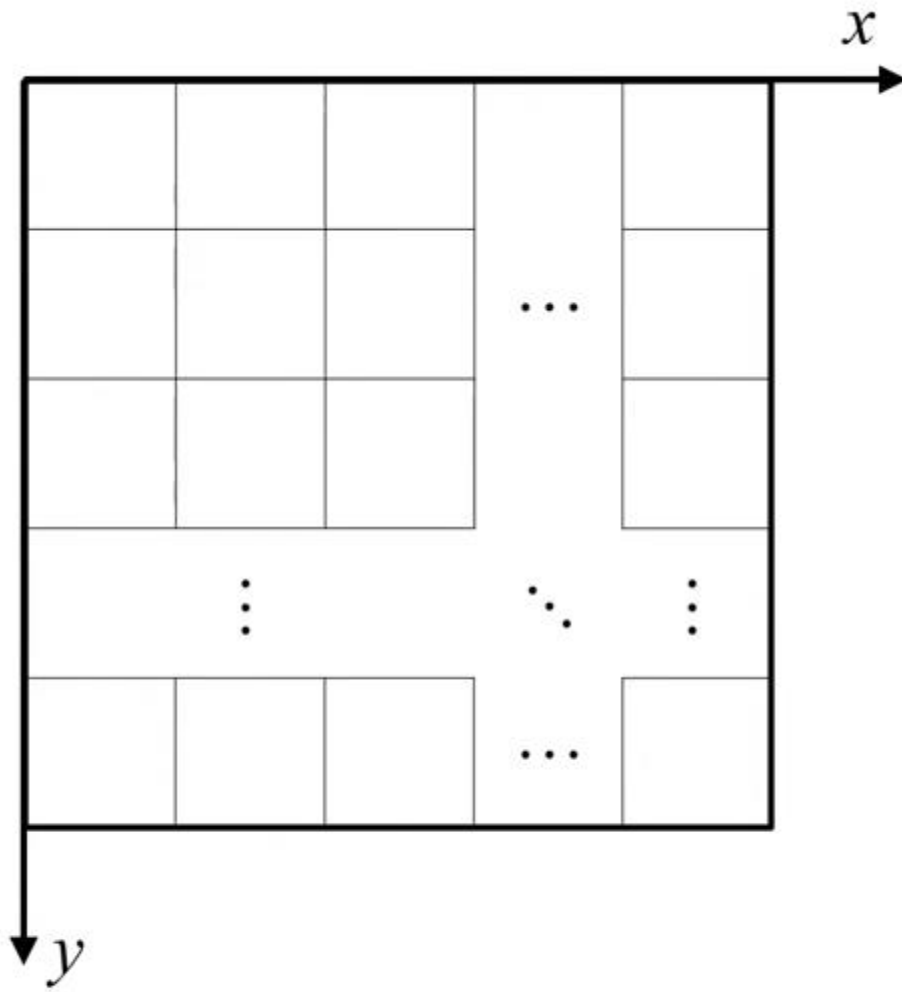
A digital image is
the set of all points inside a rectangle
over a 2D grid

An image point is called **Pixel** meaning
picture element

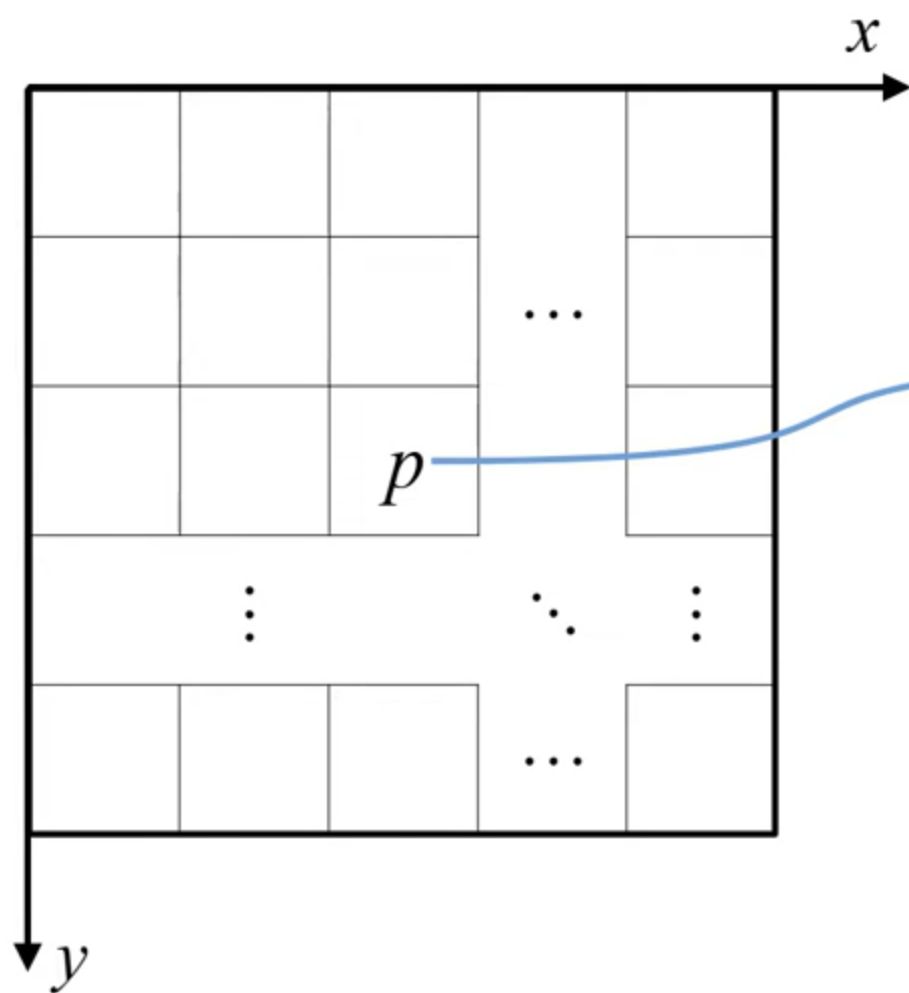


For a grayscale image, each pixel holds
brightness value

For a color image, each pixel holds
red, green and blue values



A geometric image transformation
simply moves pixels over the grid computing
new coordinates for each pixel



Pixel location is denoted as a 2D vector

$$p = \begin{bmatrix} x \\ y \end{bmatrix}$$

column coordinate

row coordinate

So transformation T takes 2D input vector
and produces 2D output vector

$$T: \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

Linear Transformation

Transformation T is linear if it meets the following three conditions
(p_1 and p_2 are vectors, c is a constant)

$$T(p_1 + p_2) = T(p_1) + T(p_2)$$

Summing two vectors before and after transformation give the same result

$$T(cp_1) = cT(p_1)$$

Scaling a vector before and after transformation give the same result

$$T(0) = 0$$

Linear transformation does not change origin

A linear transformation is implemented as matrix-vector product

$$T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

$$T(p) = Ap = p'$$

The diagram shows the equation $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$. The output vector $\begin{bmatrix} x' \\ y' \end{bmatrix}$ is enclosed in a yellow box, and the input vector $\begin{bmatrix} x \\ y \end{bmatrix}$ is also enclosed in a yellow box. The transformation matrix $\begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix}$ is enclosed in a blue box. A yellow arrow points from the output vector box to the text "Output vector p' ". A blue arrow points from the matrix box to the text "Transformation Matrix A ". A yellow arrow points from the input vector box to the text "Input vector p ".

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Output vector p' Transformation Matrix A Input vector p

A linear transformation is implemented as matrix-vector product

$$T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

$$T(p) = Ap = p'$$

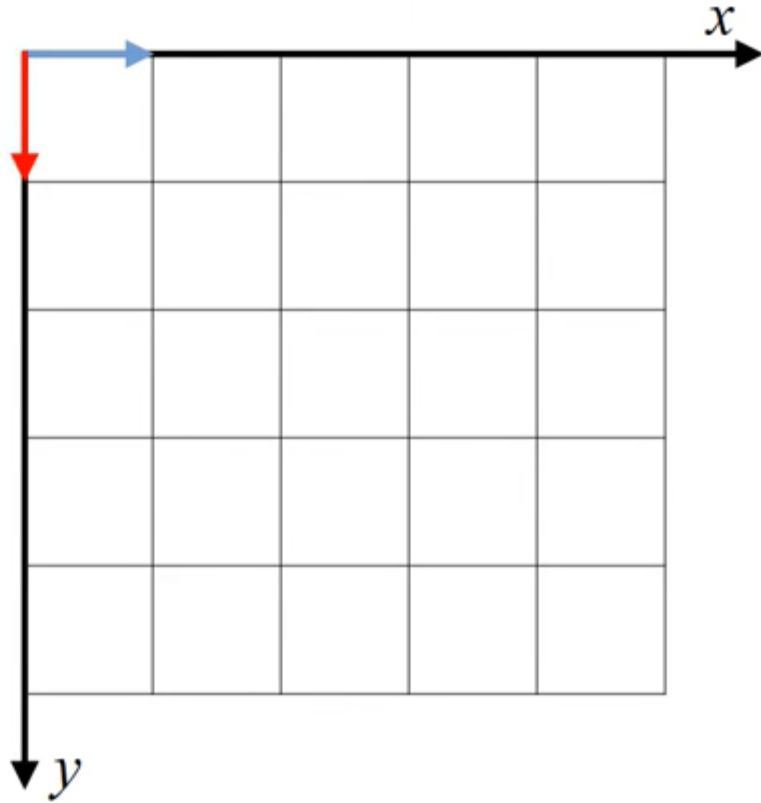
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = x \begin{bmatrix} A_{00} \\ A_{10} \end{bmatrix} + y \begin{bmatrix} A_{01} \\ A_{11} \end{bmatrix}$$

These are the basis vectors of transformed space

If transformation matrix is identity matrix

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$



If transformation matrix is as shown below

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

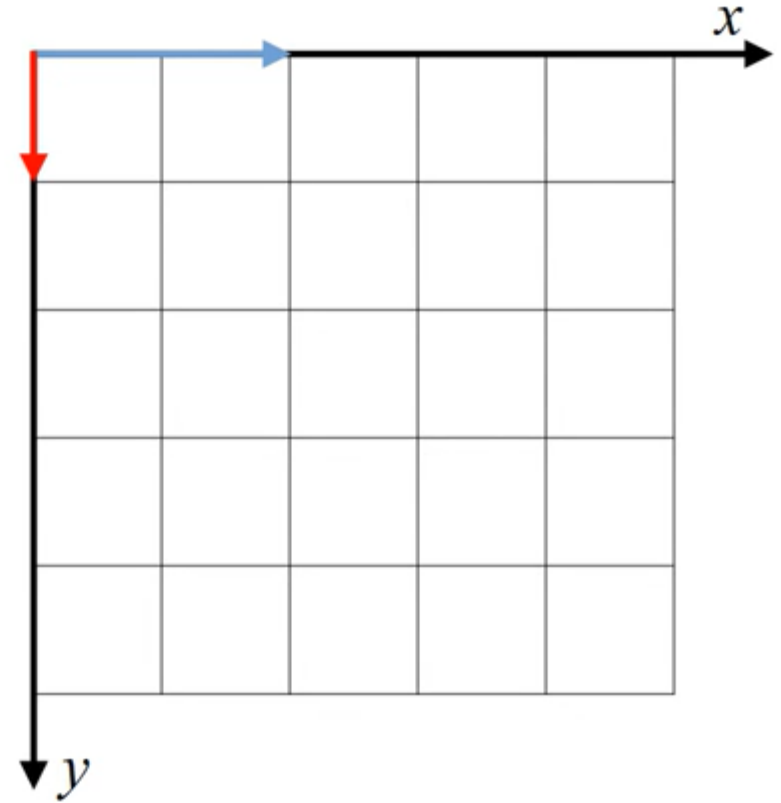


Image is scaled by 2 in horizontal direction

Sequential application of multiple linear transformations
is again a linear transformation

$$T_1: \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

$$T_1(p) = Ap$$

$$T_2: \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

$$T_2(p) = Bp$$

$$T_3: \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

$$T_3(p) = T_2(T_1(p)) = BAp = Cp$$

$$\underbrace{\begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{bmatrix} \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}}_{T_2(T_1(p))} = \underbrace{\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}}_{T_3(p)}$$

The only difference between linear and affine transformations:

Linear transformation keeps the origin same

But, affine transformation may change the origin

Linear transformation

$$T_l: \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

$$T_l(p) = Ap$$

Affine transformation

$$T_a: \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

$$T_a(p) = Ap + b$$

A is 2x2 transformation matrix

p is 2x1 coordinate vector

b is 2x1 translation vector

Linear transformation

$$T_l: \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

$$T_l(p) = Ap$$

Linear transformation is
affine transformation with zero
translation vector b

Affine transformation

$$T_a: \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

$$T_a(p) = Ap + b$$

Affine transformation is
linear transformation only when
 b is zero vector

Affine transformation equation

$$T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

$$T(p) = Ap + b = p'$$

In matrix-vector form

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$$


2D affine transform is controlled with 6 parameters, also called 6 Degrees of Freedom (DoF)

$$x' = A_{00}x + A_{01}y + b_0$$

$$y' = A_{10}x + A_{11}y + b_1$$

Zero vector (origin) may be mapped to a different point depending on b

Affine Transformation preserves lines and
ratio of lengths over each line

Affine Transformation may not preserve
angles and lengths

Parallel lines remain parallel

Homogeneous Coordinates

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$$

To compute affine transformation with above equation,

One matrix-vector multiplication and one vector addition should be performed

Single matrix-vector multiplication would be more practical and more efficient

The solution is to use homogeneous coordinates

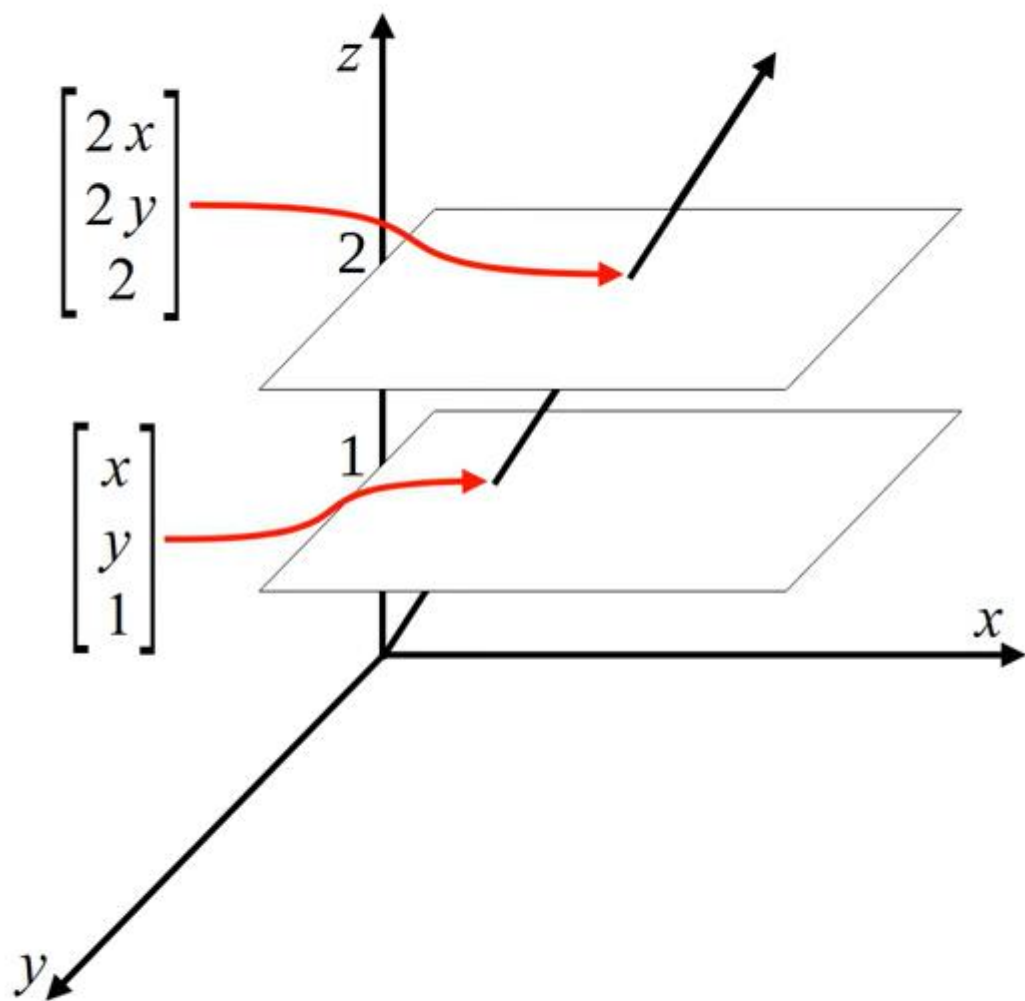
To obtain homogeneous coordinates from Cartesian coordinates,
one more dimension is added

$$\begin{bmatrix} x \\ y \end{bmatrix}$$

Cartesian
coordinates

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Homogeneous
coordinates



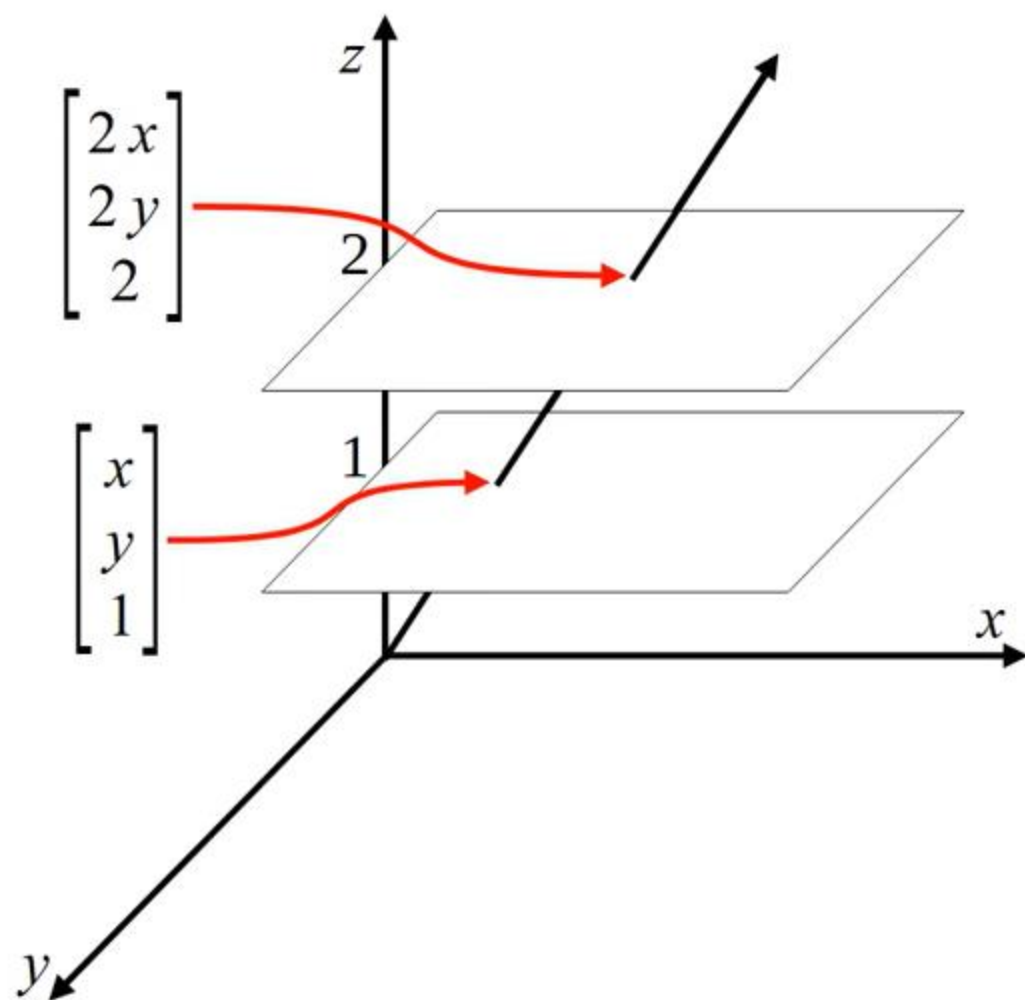
In homogeneous coordinates, a point and its scaled versions are same

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = w \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} wx \\ wy \\ w \end{bmatrix} \quad w \neq 0$$

They're the same point on different z planes

$$\begin{bmatrix} x \\ y \\ 0 \end{bmatrix}$$

If z component is zero then the point is at infinity



When going back to Cartesian coordinates

$$\begin{bmatrix} wx \\ wy \\ w \end{bmatrix}$$

Homogeneous
coordinates

$$\begin{bmatrix} \frac{wx}{w} \\ \frac{wy}{w} \\ w \end{bmatrix}$$

Cartesian
coordinates

$w \neq 0$

Scaled versions of the same point in
homogeneous coordinates correspond to
same Cartesian coordinates

Affine Transformation
in Cartesian coordinates

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$$

One more dimension is added to
input and output vectors

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Cartesian
coordinates

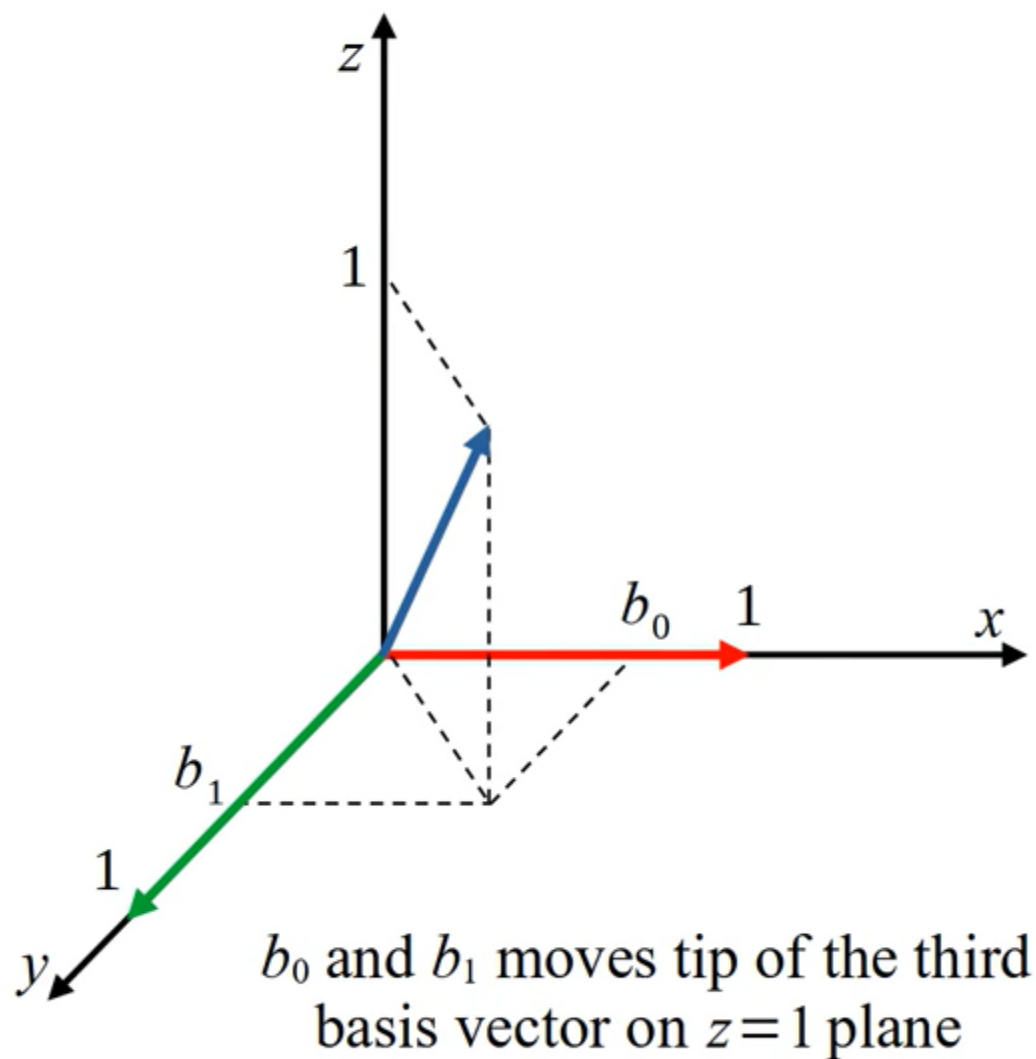
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

Homogeneous
coordinates

And b vector is moved inside
transformation matrix or
augmented to transformation matrix

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} & b_0 \\ A_{10} & A_{11} & b_1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Affine Transformation
in homogeneous coordinates



This matrix is a linear transformation matrix in 3D

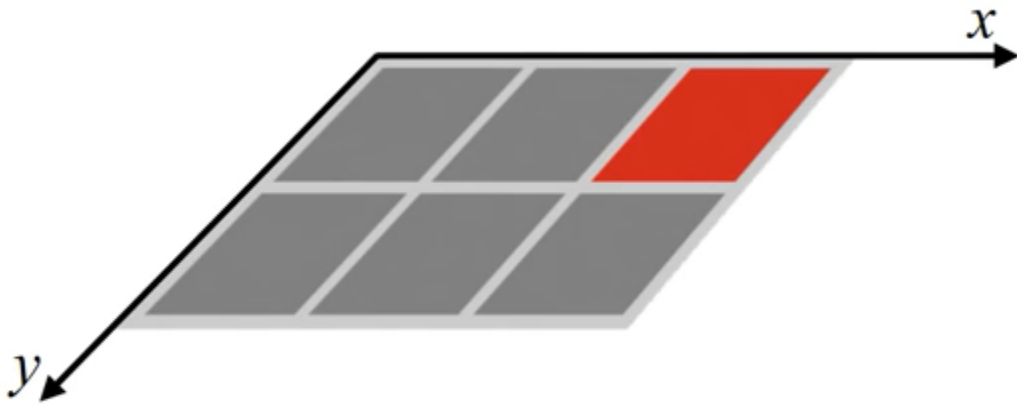
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} & b_0 \\ A_{10} & A_{11} & b_1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

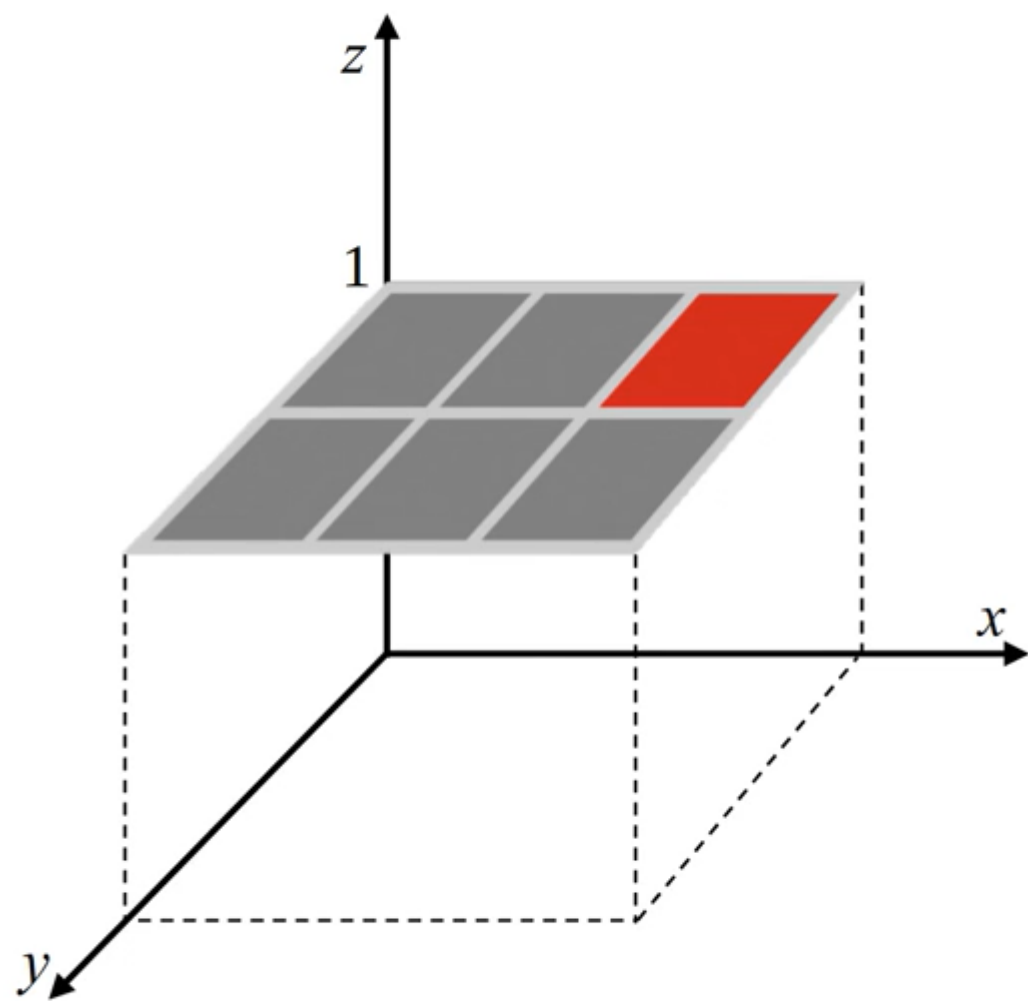
Then this column is the third basis vector of transformed vector space

And what b_0 and b_1 do is to change the orientation of that basis vector

This is our input image in Cartesian coordinates

Image scale in this diagram is not relevant,
just for illustration



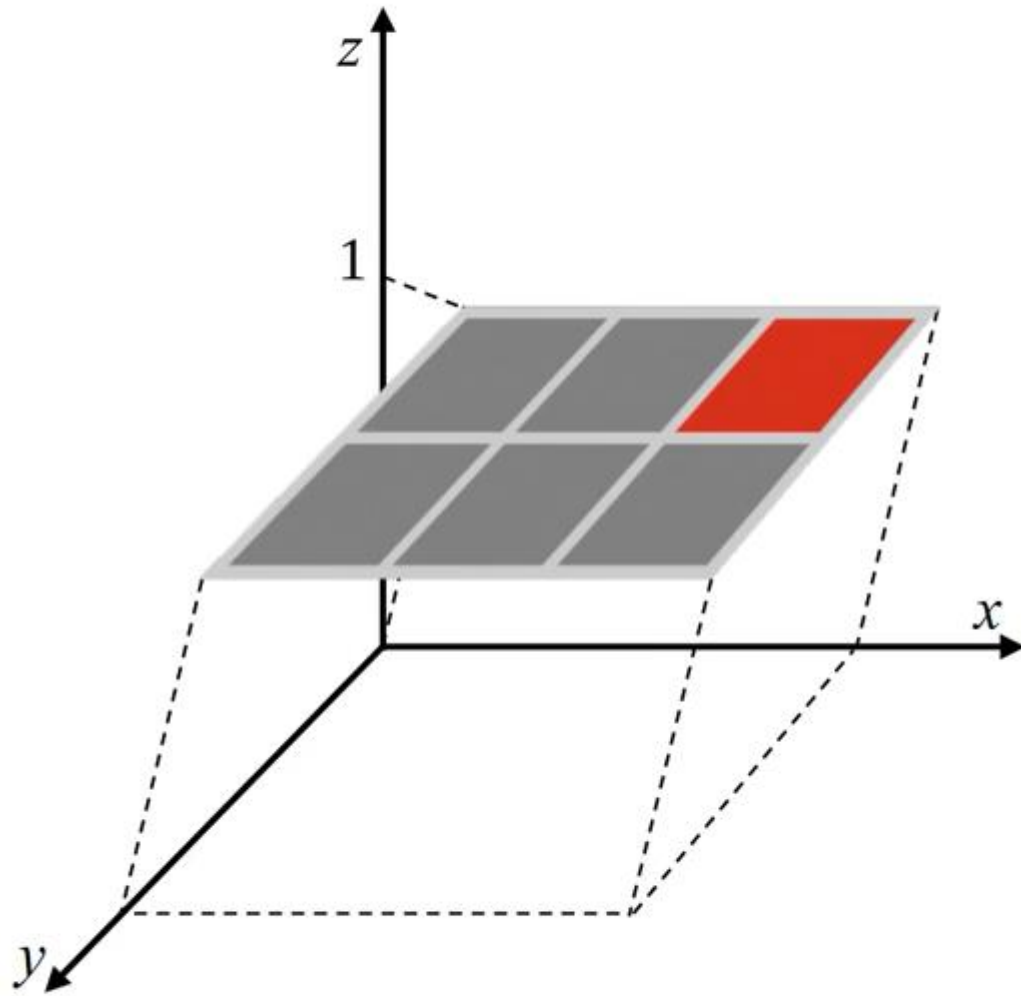


When we move from Cartesian to homogeneous coordinates

$$\begin{bmatrix} x \\ y \end{bmatrix} \longrightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D image is moved to $z=1$ plane in 3D

So, 2D image is embedded in 3D

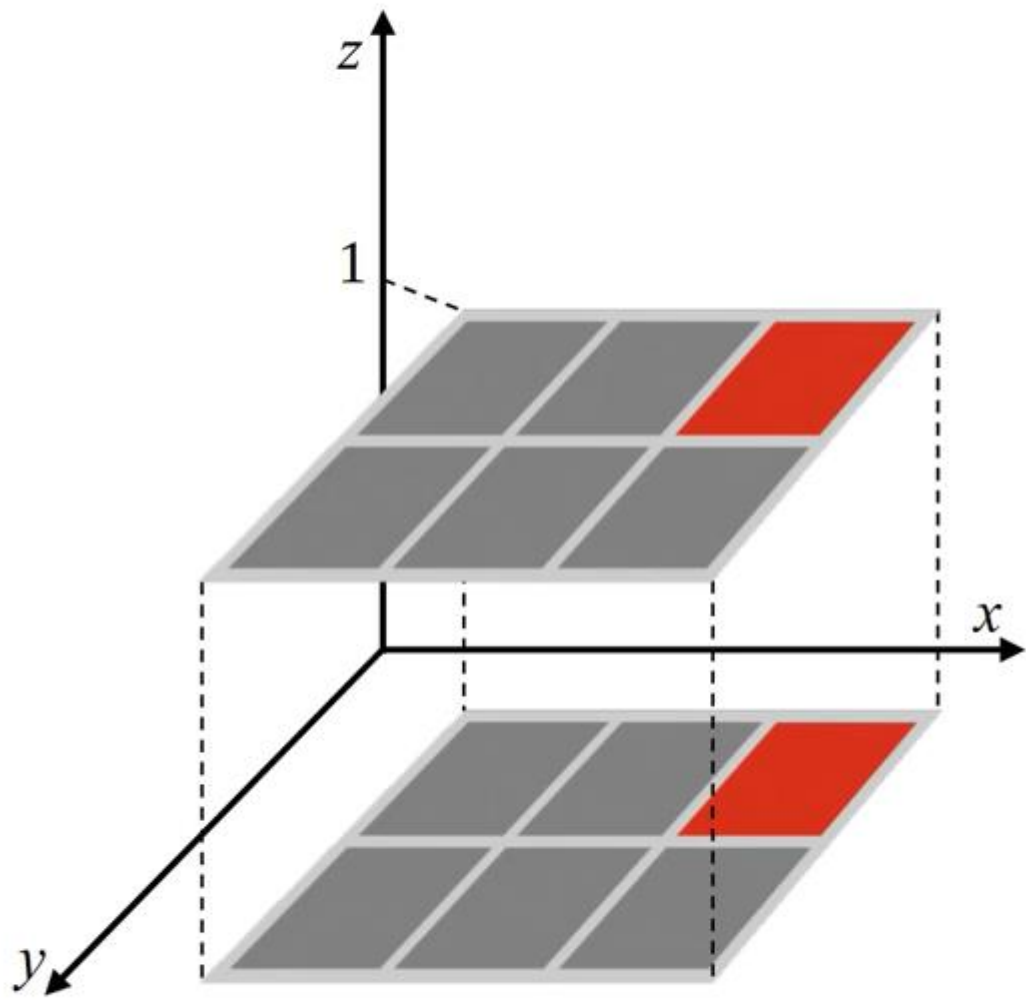


This matrix is a linear transformation matrix in 3D

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} & b_0 \\ A_{10} & A_{11} & b_1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

b_0 and b_1 shear 3D space parallel to xy plane

Translation of image over 2D $z=1$ plane is the result of shear in 3D space



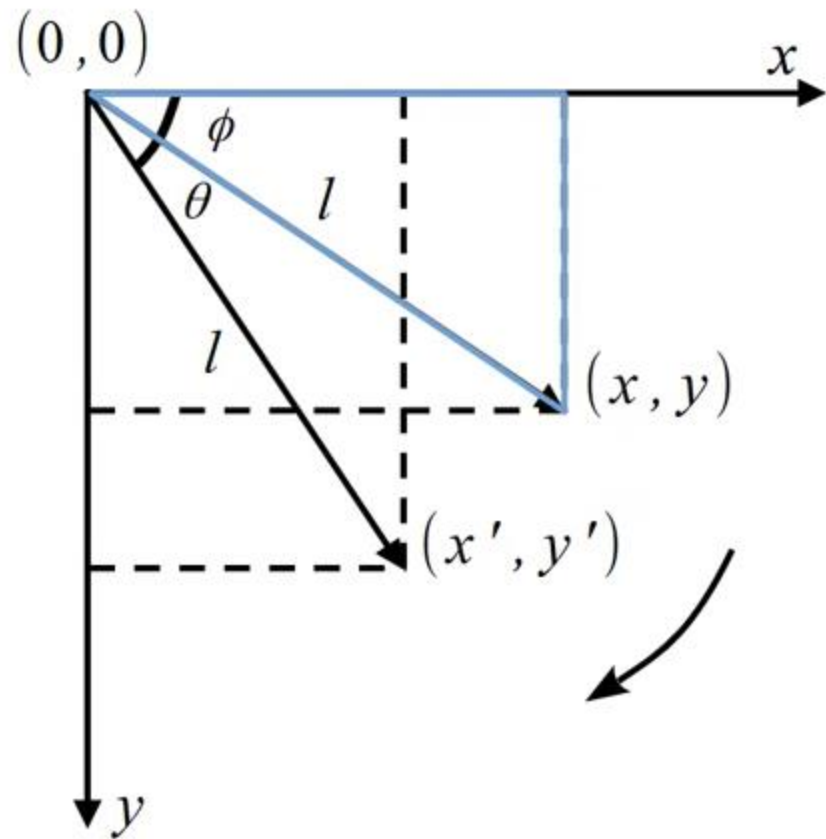
When going back to
Cartesian coordinates

$$\frac{x'}{z'} = \frac{x'}{1} = x'$$

$$\frac{y'}{z'} = \frac{y'}{1} = y'$$

Image Rotation

Rotation Around Origin
with Linear Transformation



Let's start rotating pixel at (x, y)
and derive the rotation matrix

θ is clockwise rotation angle

Center of rotation is $(0, 0)$

Vector length l does not change

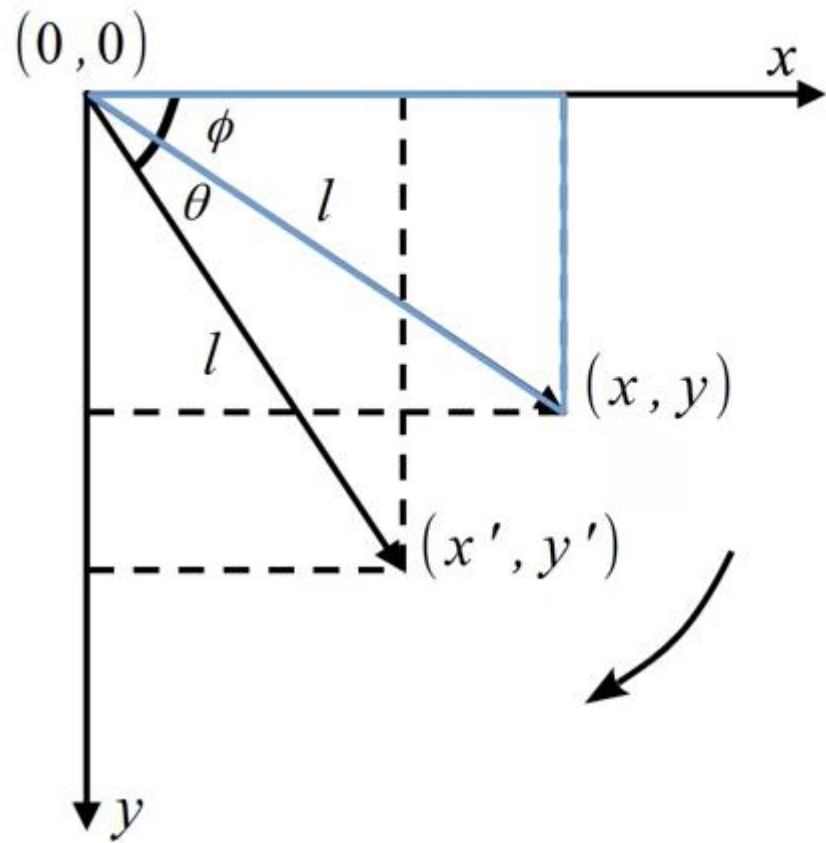
$$x' = l \cos(\phi + \theta)$$

$$x' = l \cos \phi \cos \theta - l \sin \phi \sin \theta$$

$$l \cos \phi = x$$

$$l \sin \phi = y$$

$$x' = x \cos \theta - y \sin \theta$$



Let's start rotating pixel at (x, y)
and derive the rotation matrix

θ is clockwise rotation angle

Center of rotation is $(0, 0)$

Vector length l does not change

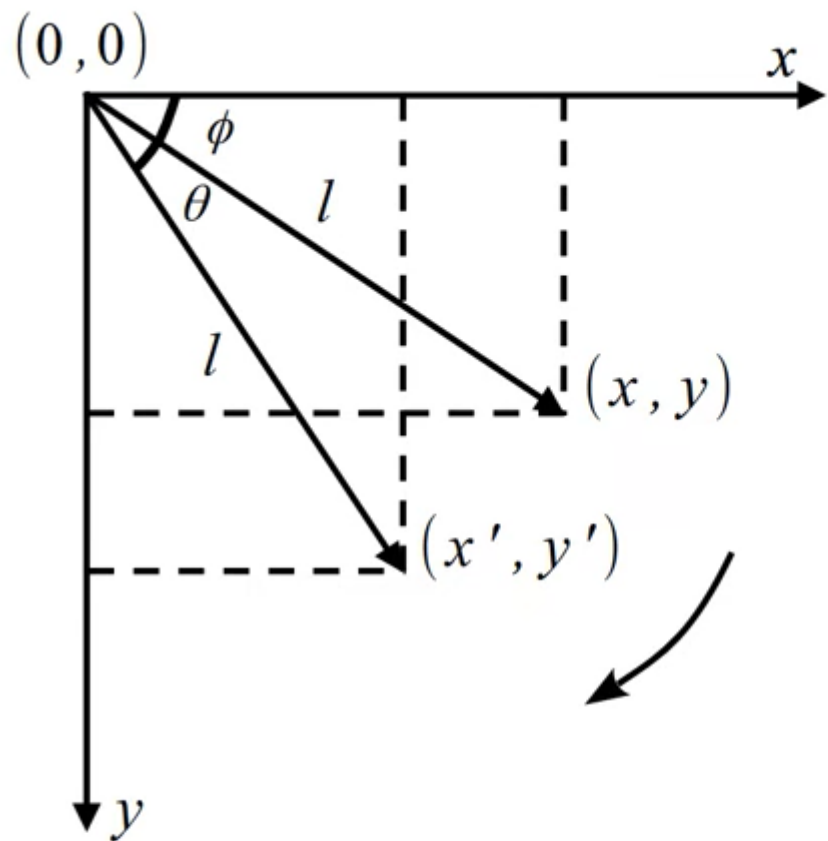
$$y' = l \sin(\phi + \theta)$$

$$y' = l \sin \phi \cos \theta + l \cos \phi \sin \theta$$

$$l \sin \phi = y$$

$$l \cos \phi = x$$

$$y' = x \sin \theta + y \cos \theta$$



$$x' = x \cos \theta - y \sin \theta$$

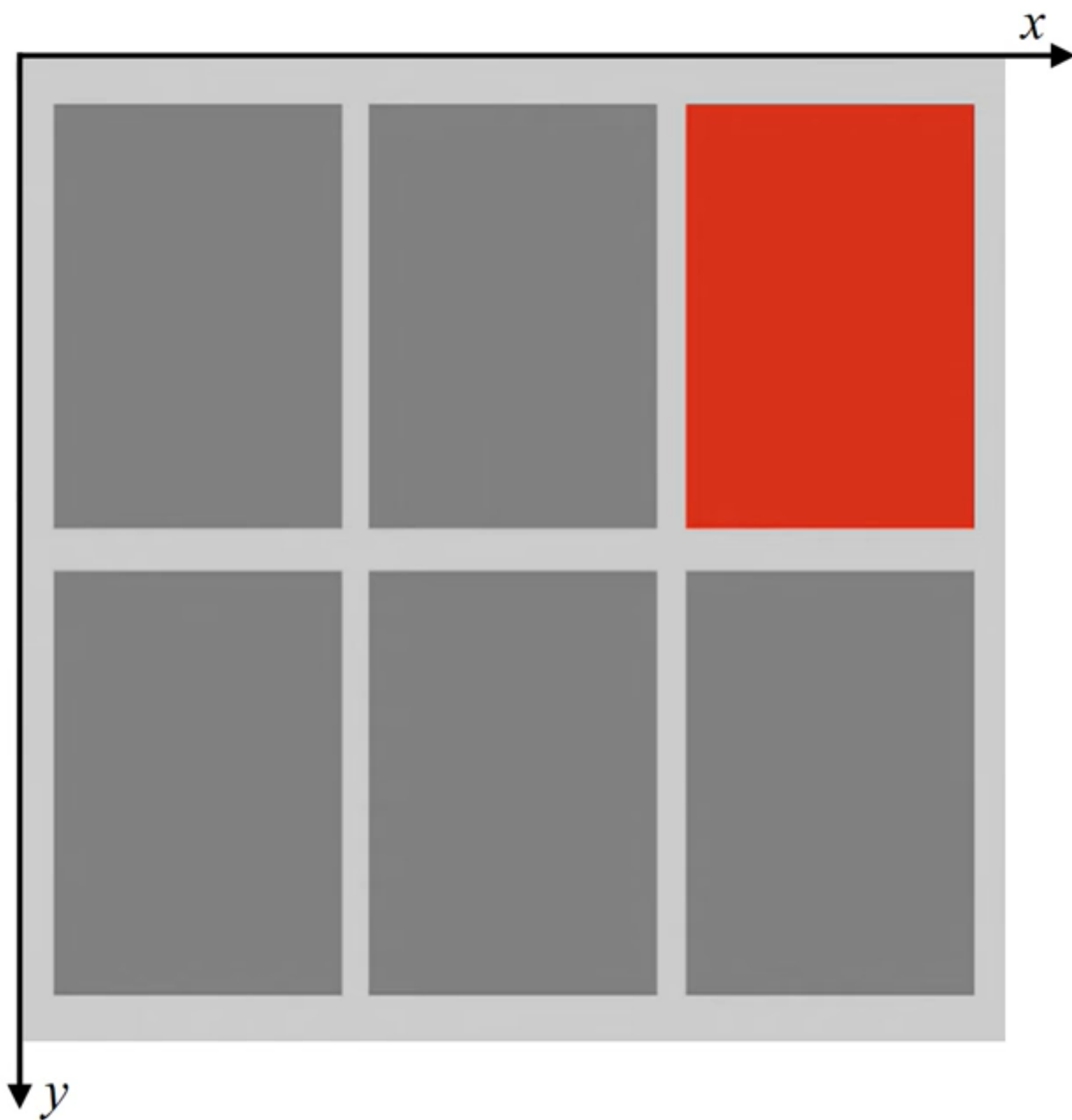
$$y' = x \sin \theta + y \cos \theta$$

As matrix equation

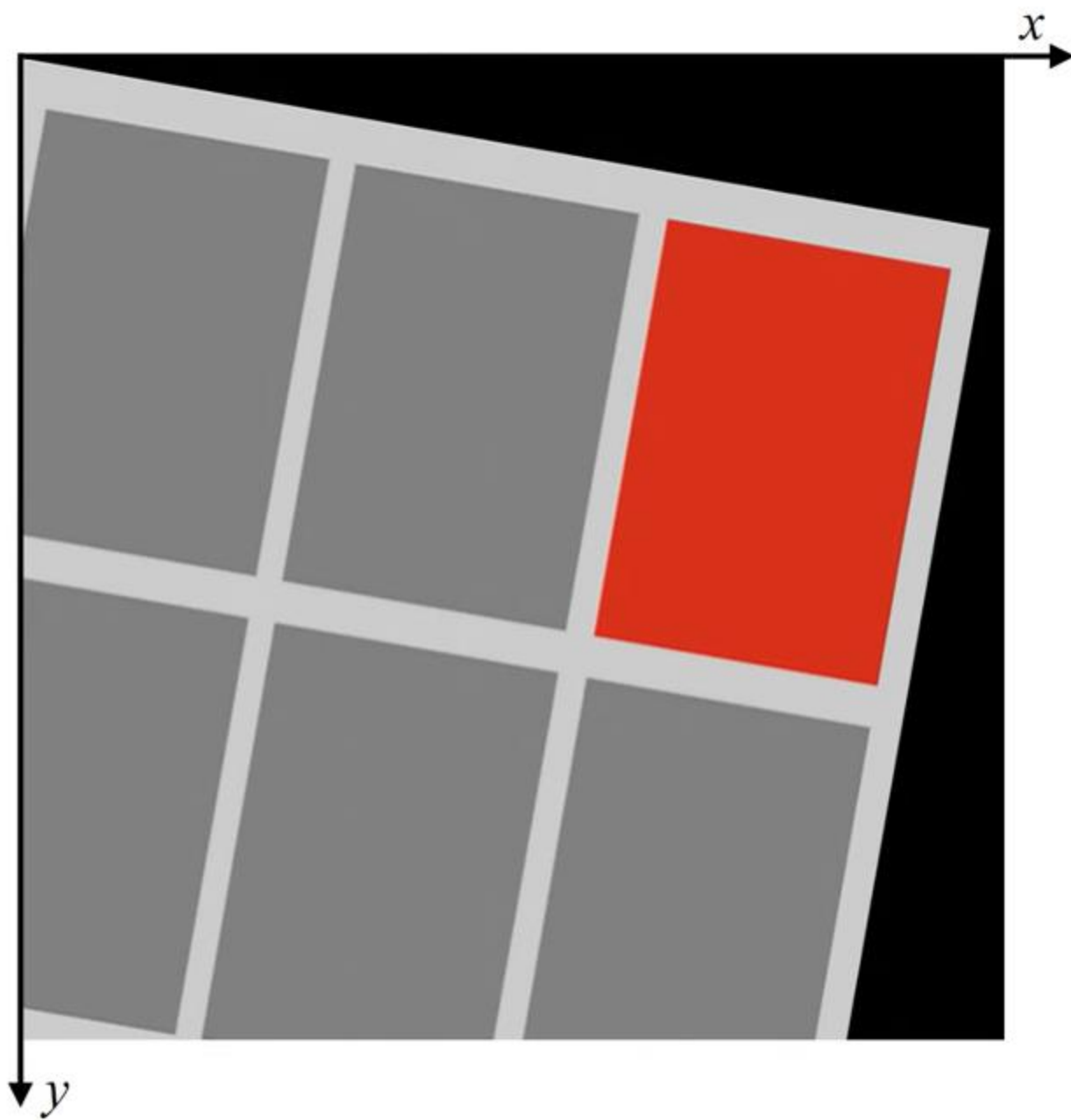
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (\text{clockwise})$$

For counterclockwise rotation,
transpose of the matrix is used

$$\longrightarrow \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (\text{counterclockwise})$$

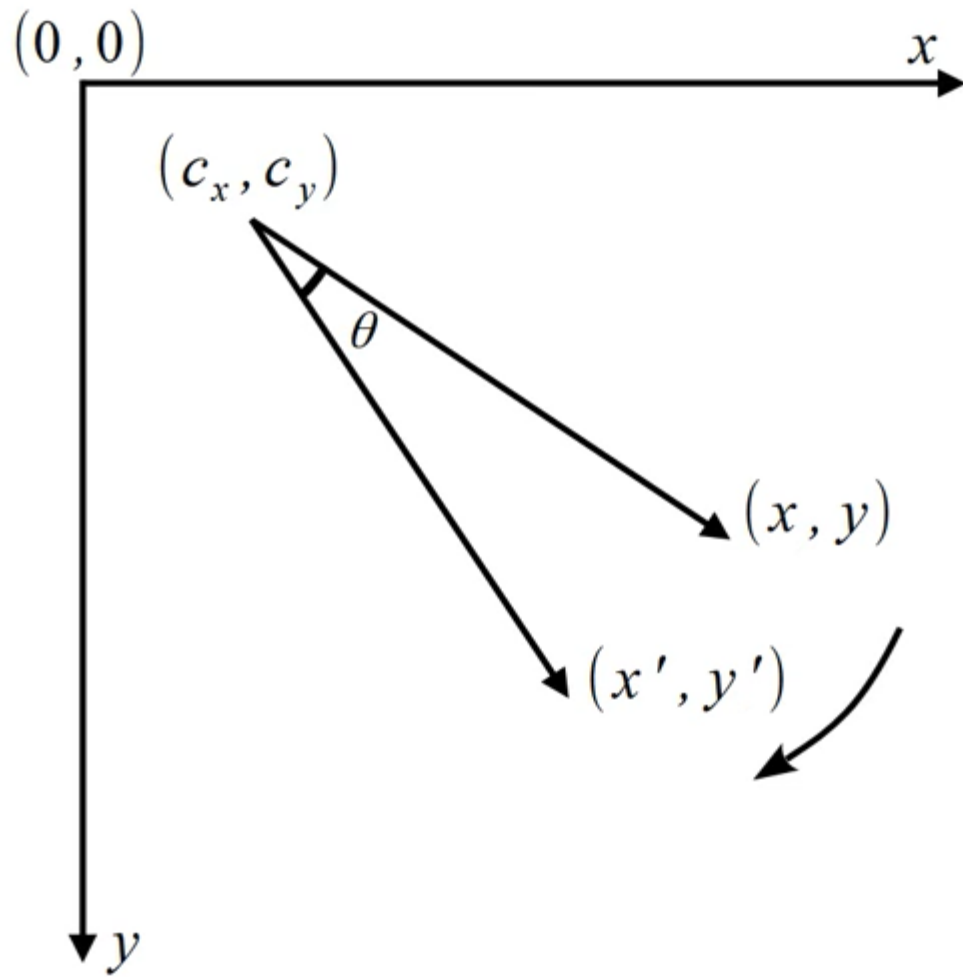


If θ is 10° and rotation
is clockwise



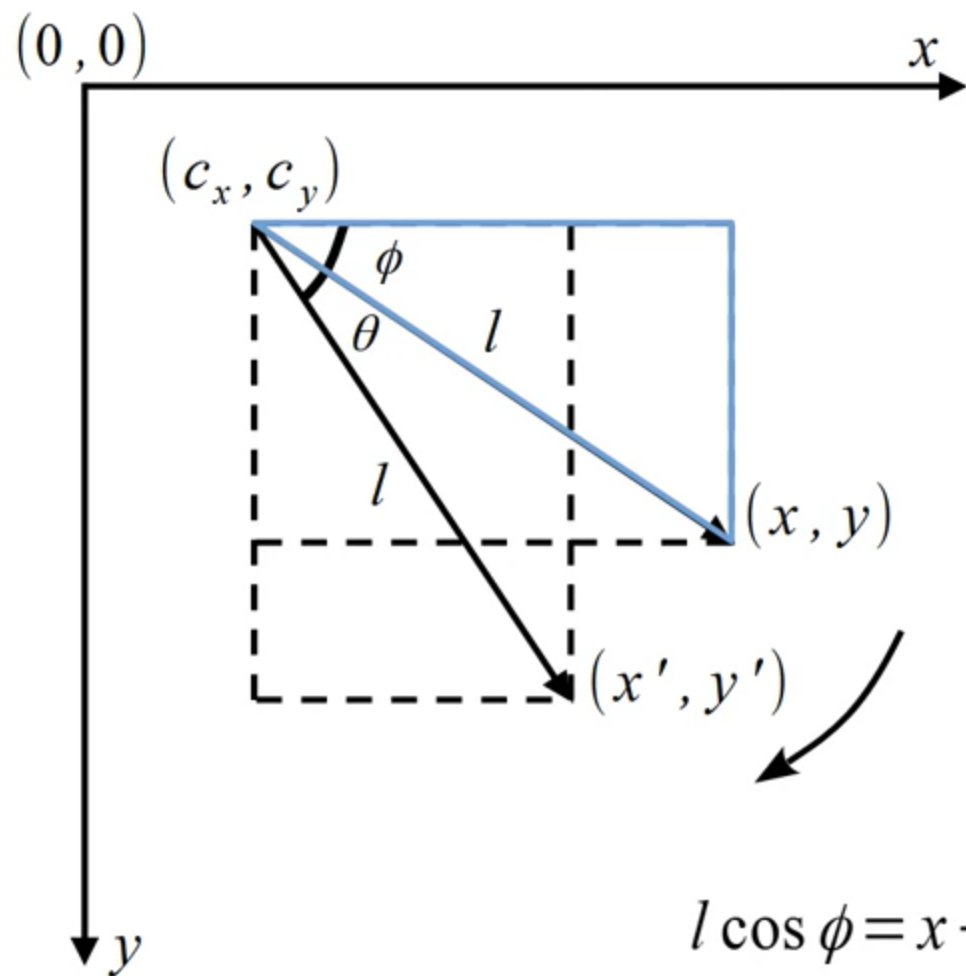
If θ is 10° and rotation
is clockwise

Rotation Around Any Point
with Affine Transformation



Let's start rotating pixel at (x, y)
and derive the rotation matrix

Center of rotation is (c_x, c_y)
 θ is clockwise rotation angle



Let's start rotating pixel at (x, y)
and derive the rotation matrix

Center of rotation is (c_x, c_y)

θ is clockwise rotation angle

Vector length l does not change

$$x' = c_x + l \cos(\phi + \theta)$$

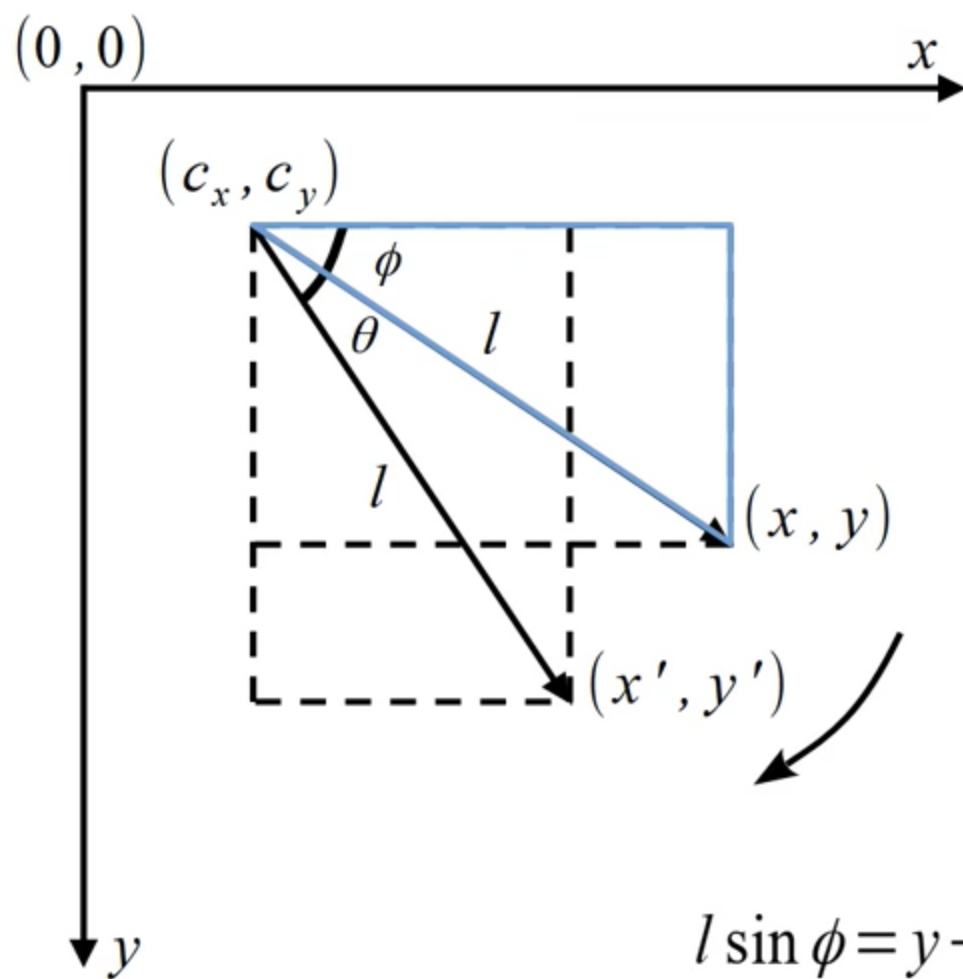
$$x' = c_x + l \cos \phi \cos \theta - l \sin \phi \sin \theta$$

$$l \cos \phi = x - c_x$$

$$l \sin \phi = y - c_y$$

$$x' = c_x + (x - c_x) \cos \theta - (y - c_y) \sin \theta$$

$$x' = x \cos \theta - y \sin \theta + c_x (1 - \cos \theta) + c_y \sin \theta$$



Let's start rotating pixel at (x, y)
and derive the rotation matrix

Center of rotation is (c_x, c_y)

θ is clockwise rotation angle

Vector length l does not change

$$y' = c_y + l \sin(\phi + \theta)$$

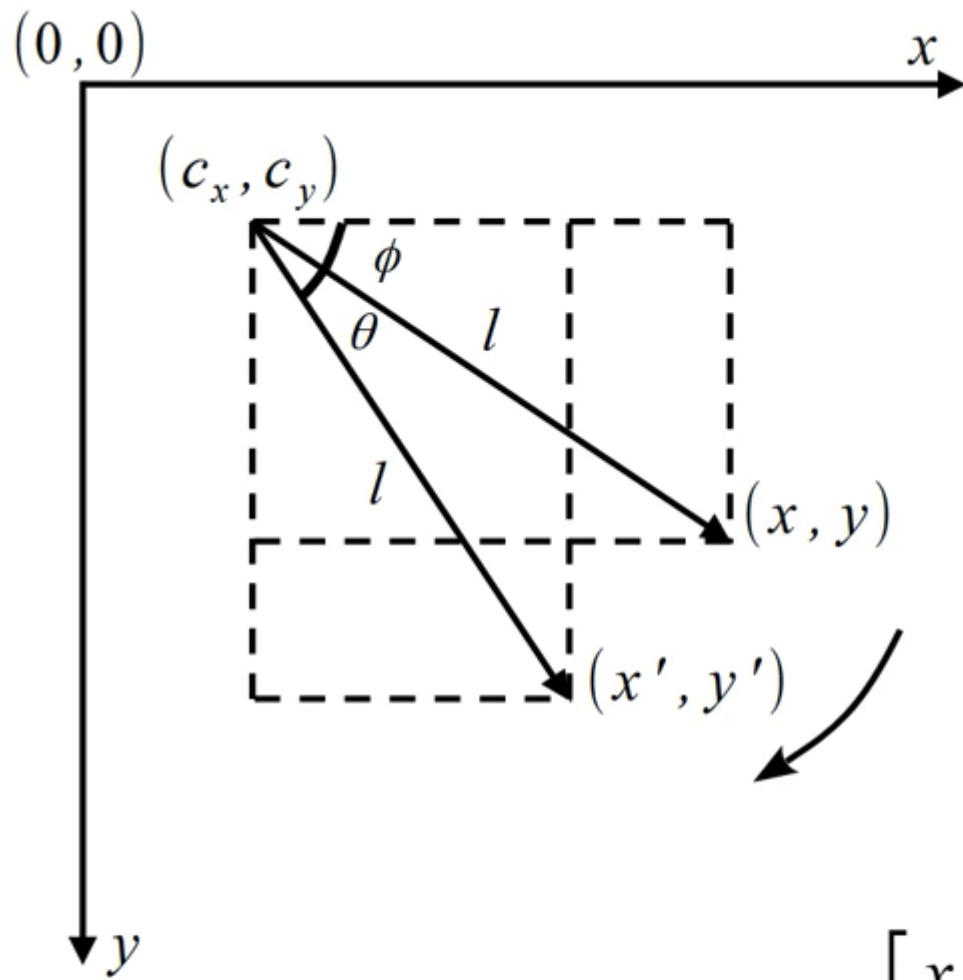
$$y' = c_y + \boxed{l \sin \phi} \cos \theta + \boxed{l \cos \phi} \sin \theta$$

$$l \sin \phi = y - c_y$$

$$l \cos \phi = x - c_x$$

$$y' = c_y + (y - c_y) \cos \theta + (x - c_x) \sin \theta$$

$$y' = y \cos \theta + x \sin \theta + c_y(1 - \cos \theta) - c_x \sin \theta$$



$$x' = x \cos \theta - y \sin \theta + c_x (1 - \cos \theta) + c_y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta + c_y (1 - \cos \theta) - c_x \sin \theta$$

As matrix-vector multiplication

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & c_x (1 - \cos \theta) + c_y \sin \theta \\ \sin \theta & \cos \theta & c_y (1 - \cos \theta) - c_x \sin \theta \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & c_x(1 - \cos \theta) + c_y \sin \theta \\ \sin \theta & \cos \theta & c_y(1 - \cos \theta) - c_x \sin \theta \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$


In fact, this column and this vector remind of something

With a slight modification, the equation can be expressed
in homogeneous coordinates

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & c_x(1 - \cos \theta) + c_y \sin \theta \\ \sin \theta & \cos \theta & c_y(1 - \cos \theta) - c_x \sin \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & c_x(1 - \cos \theta) + c_y \sin \theta \\ \sin \theta & \cos \theta & c_y(1 - \cos \theta) - c_x \sin \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

With a single matrix-vector multiplication,

Image is first translated by $(-c_x, -c_y)$

Rotated θ° in clockwise direction around origin

Then translated back to its previous coordinates

Translation back

Rotation

Translation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & c_x \\ 0 & 1 & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -c_x \\ 0 & 1 & -c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

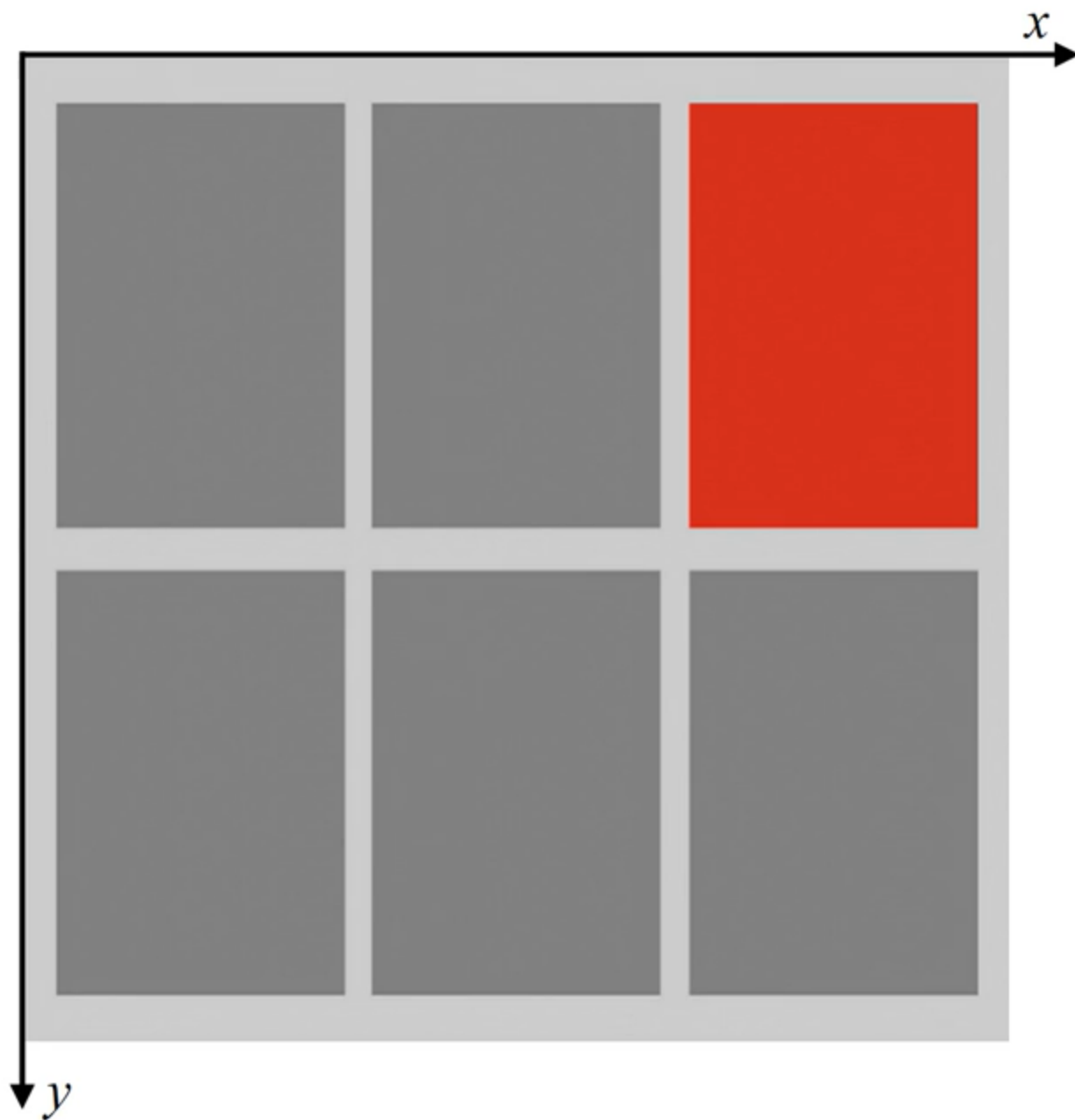
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & c_x \\ 0 & 1 & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & -c_x \cos \theta + c_y \sin \theta \\ \sin \theta & \cos \theta & -c_x \sin \theta - c_y \cos \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & c_x(1 - \cos \theta) + c_y \sin \theta \\ \sin \theta & \cos \theta & c_y(1 - \cos \theta) - c_x \sin \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

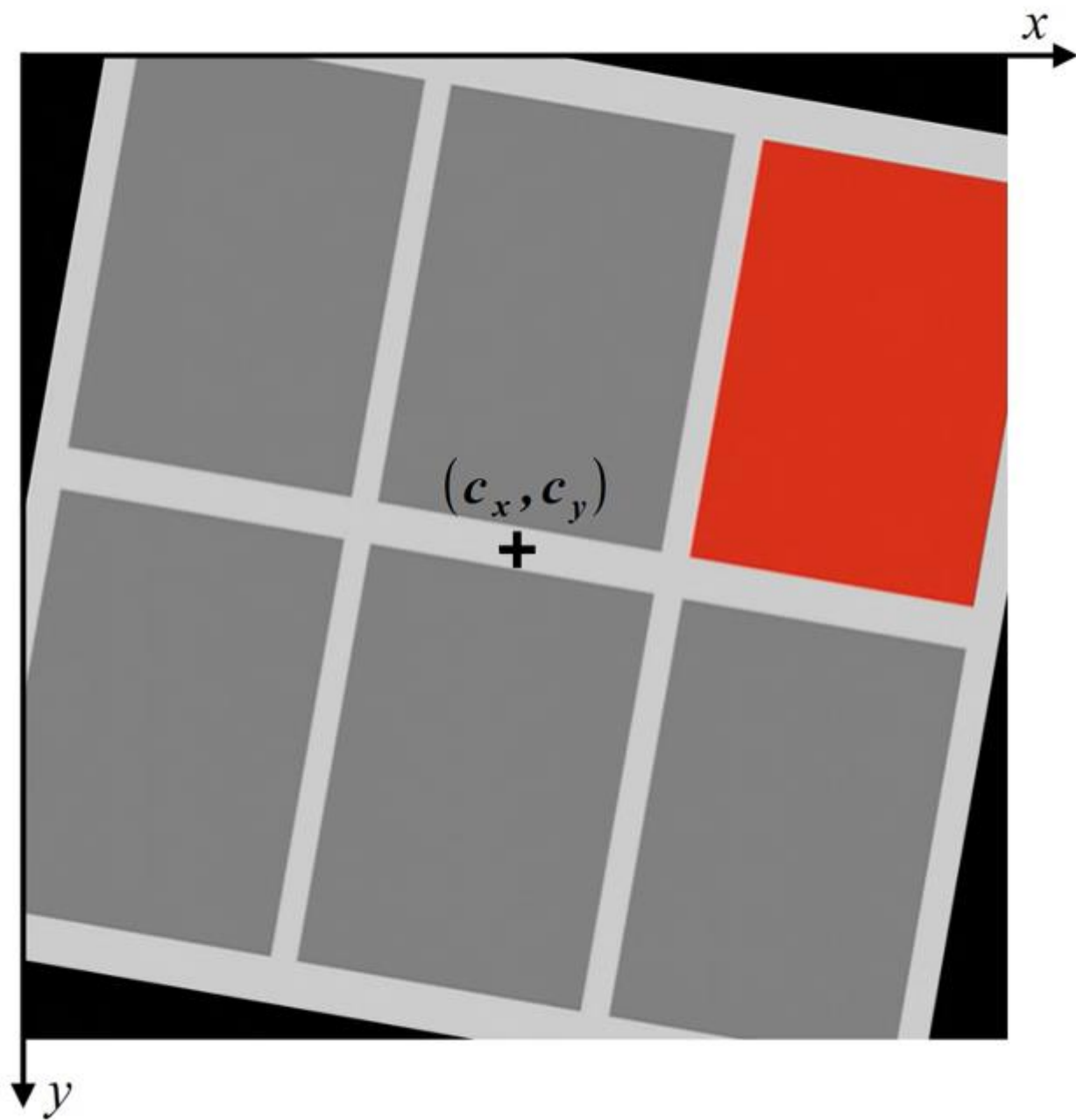
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & c_x(1 - \cos \theta) + c_y \sin \theta \\ \sin \theta & \cos \theta & c_y(1 - \cos \theta) - c_x \sin \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

If center of rotation is origin, $(c_x, c_y) = (0, 0)$

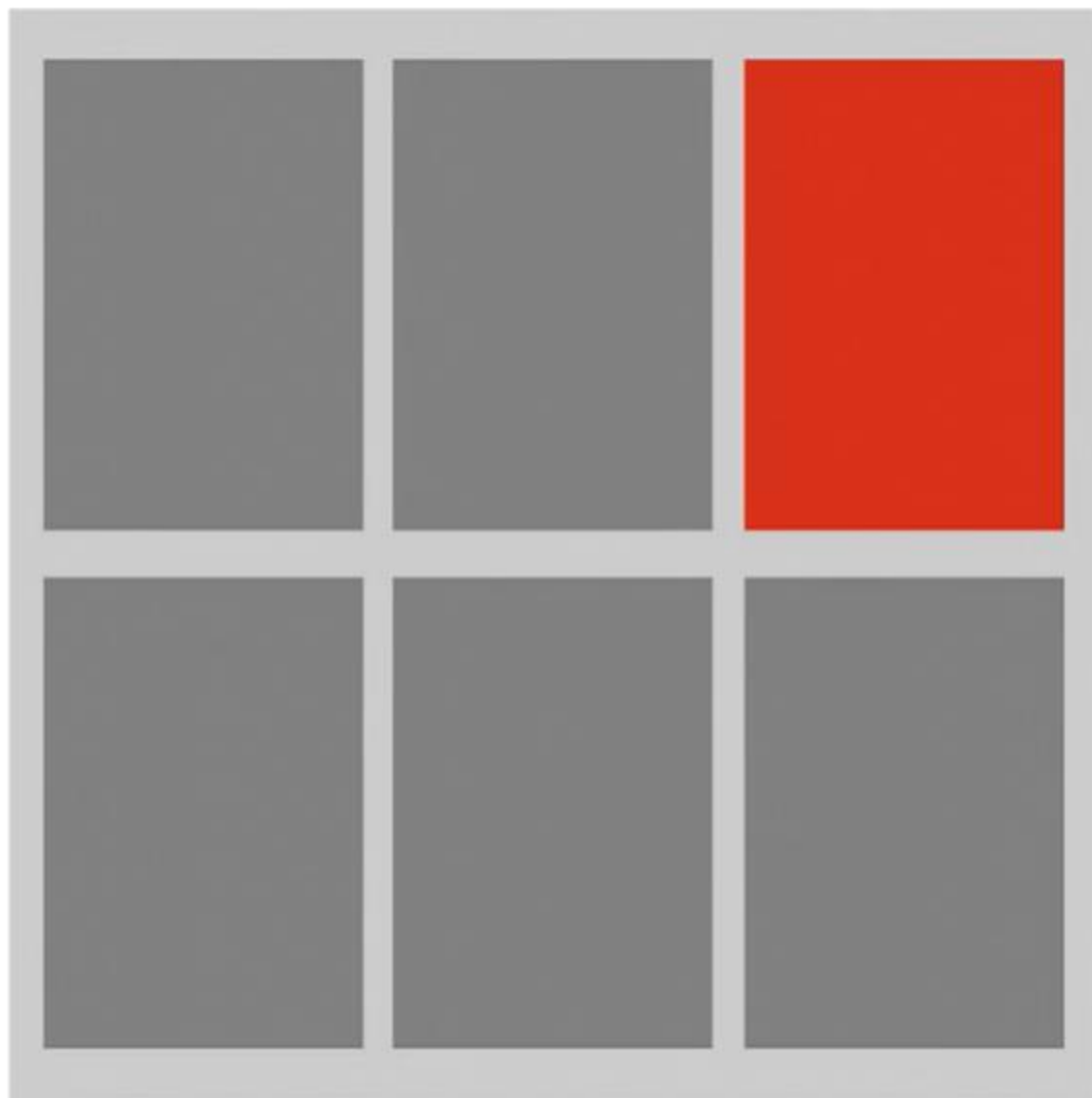
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



For images, center of rotation is
generally image center

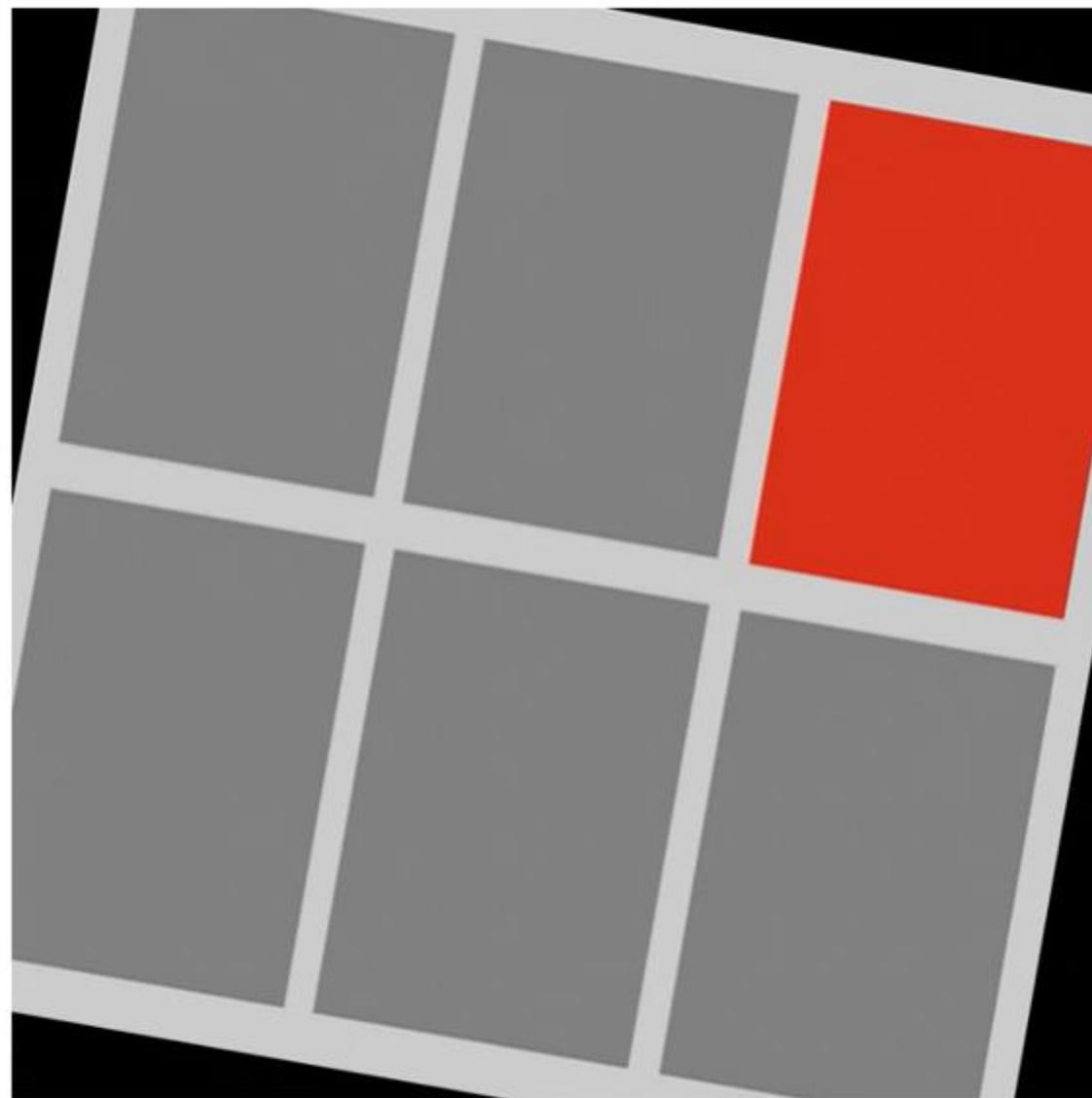


If θ is 10° and rotation
is clockwise



Original Image

$\theta = 10^\circ$ clockwise



Rotated Image

Image Scaling

Linear Transformation equation for scaling (resizing)

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \alpha & 0 \\ 0 & \beta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$x' = \alpha x$$

$$y' = \beta y$$

α and β are scale parameters for horizontal and vertical directions respectively

Linear Transformation equation for scaling (resizing)

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \alpha & 0 \\ 0 & \beta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

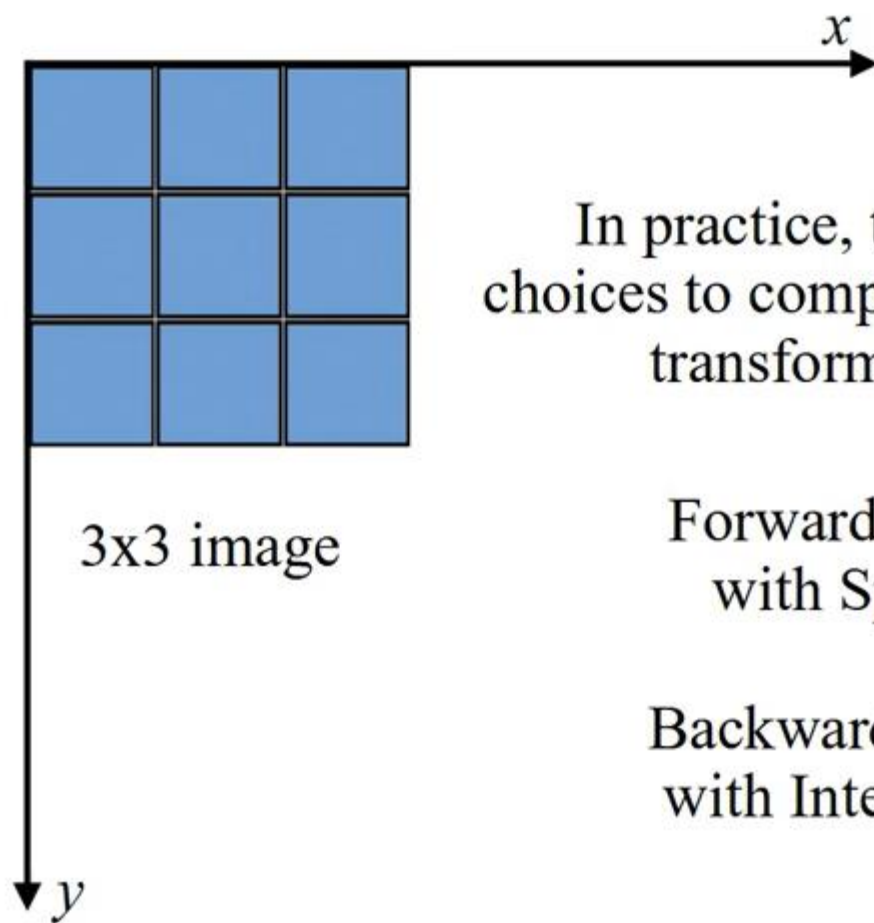
$$x' = \alpha x$$

$$y' = \beta y$$

If α is equal to β , then transformation is
isotropic (uniform) scale

This is the equation to scale in
both directions by 2

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

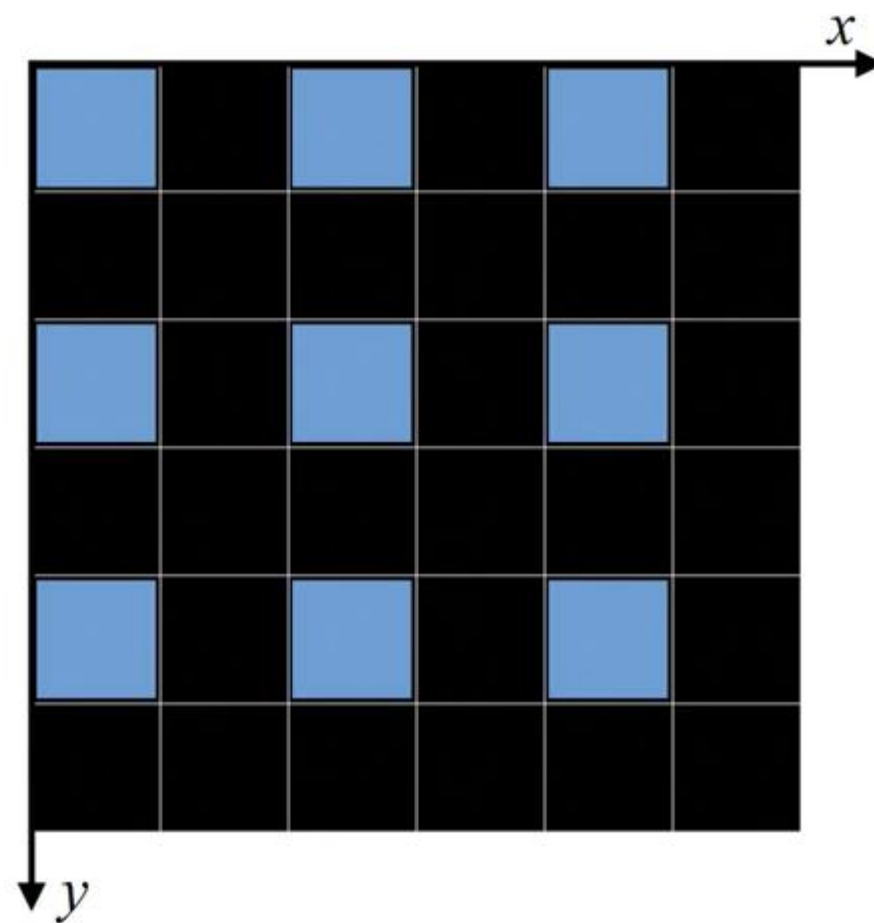


3x3 image

In practice, there are two
choices to compute the pixels of
transformed image

Forward Warping
with Splatting

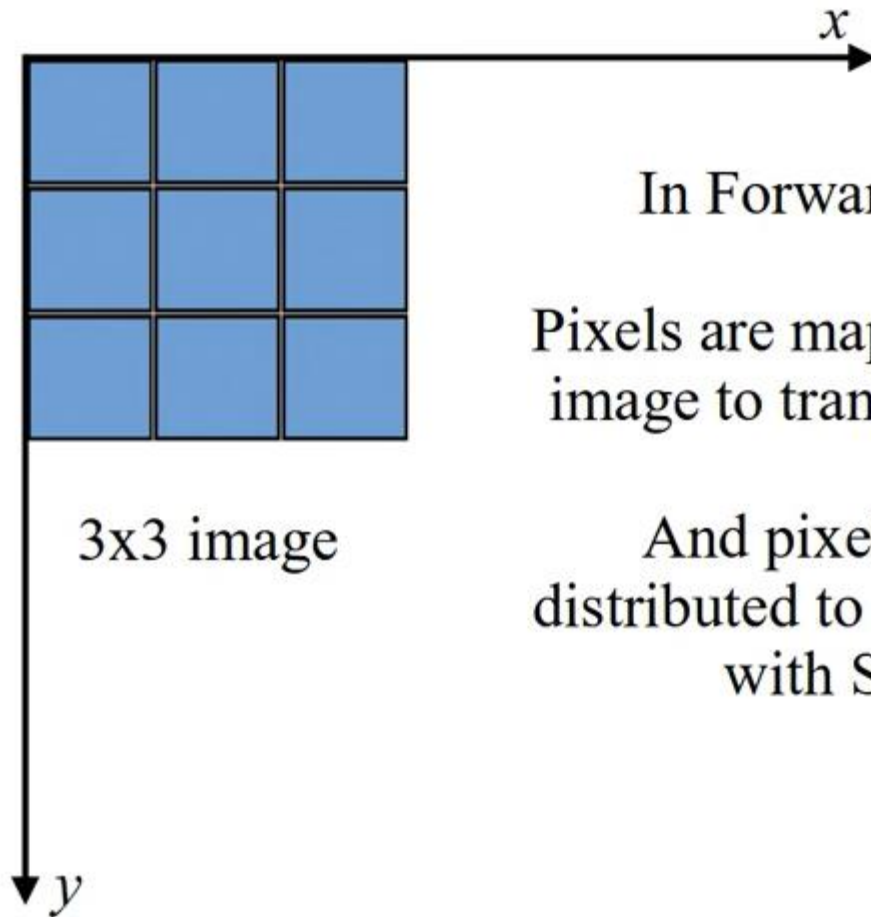
Backward Warping
with Interpolation



75% of the pixels remain empty

This is the equation to scale in
both directions by 2

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

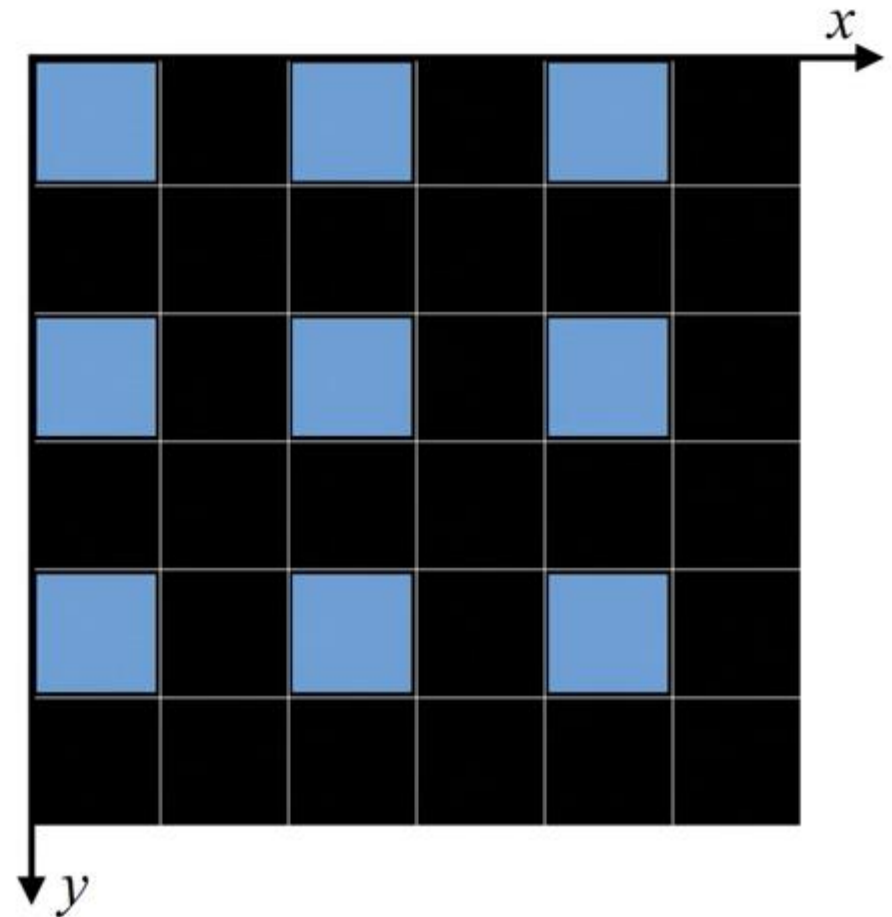


3x3 image

In Forward Warping,

Pixels are mapped from input
image to transformed image

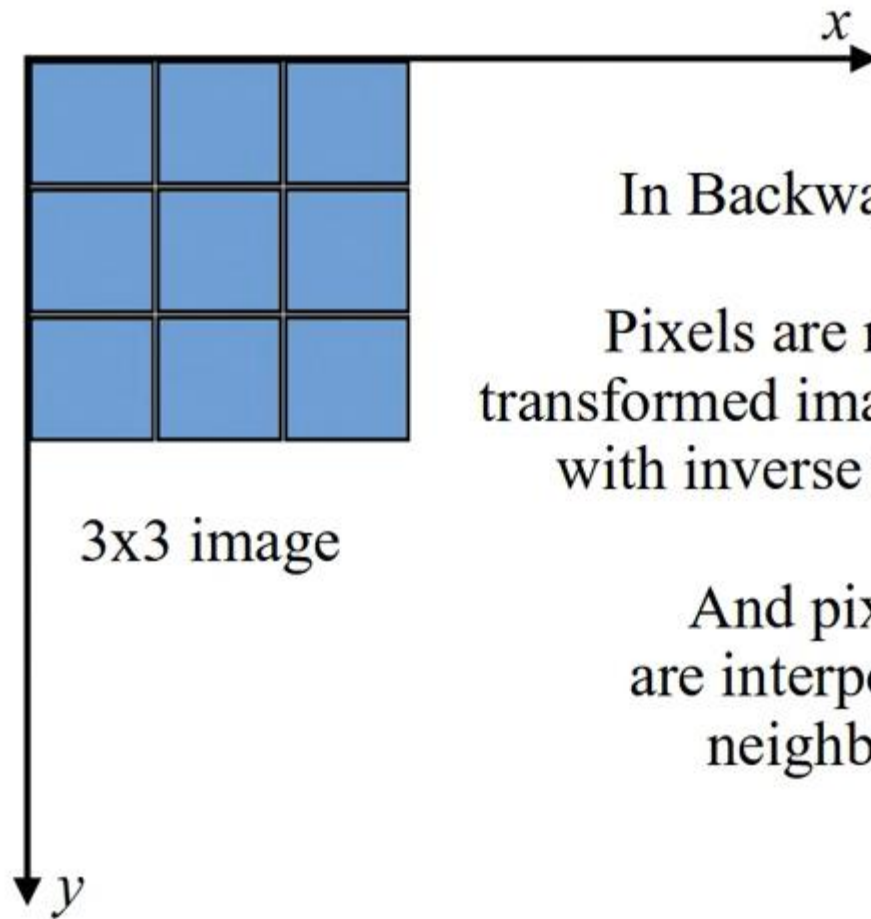
And pixel values are
distributed to neighbor pixels
with Splatting



75% of the pixels remain empty

This is the equation to scale in
both directions by 2

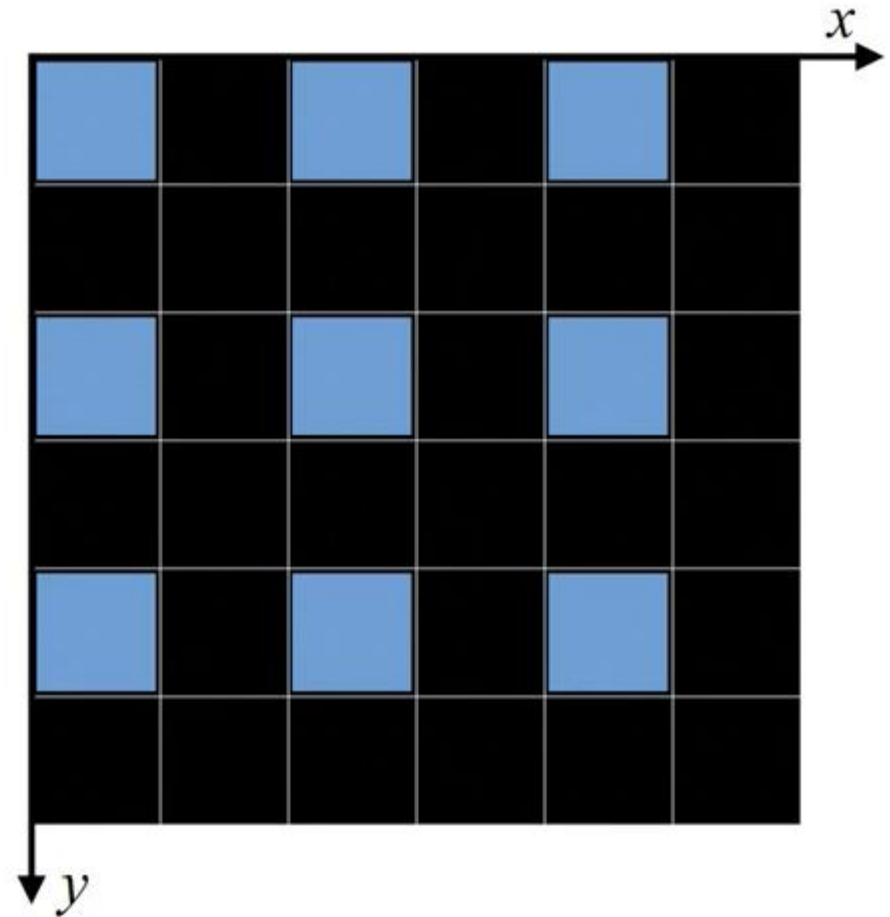
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



3x3 image

In Backward Warping,
Pixels are mapped from
transformed image to input image
with inverse transformation

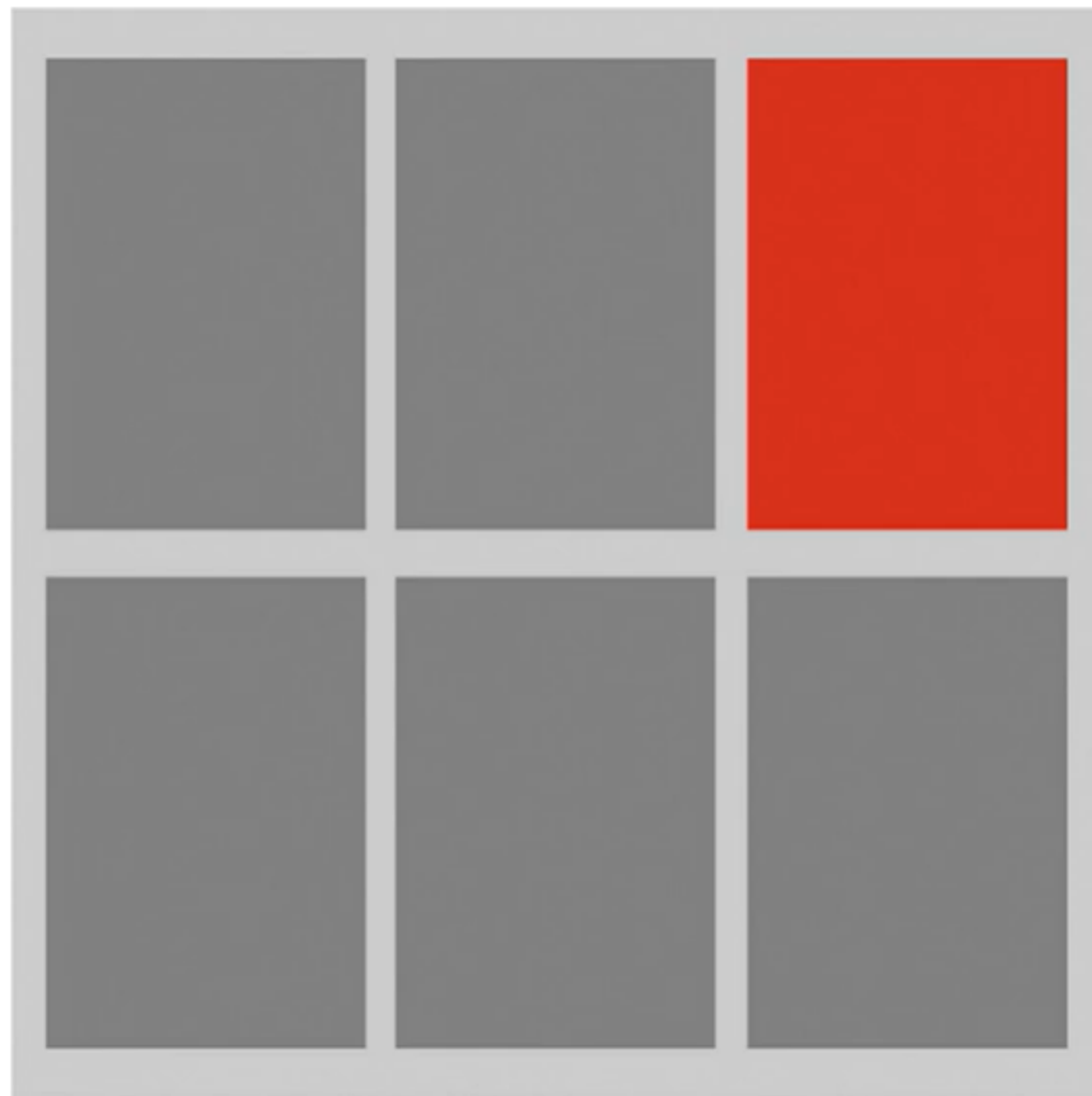
And pixel values
are interpolated using
neighbor pixels



75% of the pixels remain empty



Original Image



Scaled (Resized) Image

References

- https://www.youtube.com/watch?v=B1zCAibMcCM&list=PLcEc12xiYZ7r24dJWP4GYwwXUgGS-wl4_&index=2
- https://www.youtube.com/watch?v=uEaevJzZYhc&list=PLcEc12xiYZ7r24dJWP4GYwwXUgGS-wl4_&index=9